中国科学技术大学

硕士学位论文

面向 DRAM 近存架构的注意力计算数据流编译和设计空间探索

作者姓名： 作者姓名

学科专业： 专业

导　　师： 导师

完成时间： 二〇二六年三月三十日

University of Science and Technology of China

A dissertation for master's degree

Dataflow Compilation and Design Space Exploration for Attention on DRAM Near-Memory Architectures

Author: Author Name

Speciality: Speciality

Supervisor: Supervisor Name

Completion date: March 30, 2026

中国科学技术大学学位论文原创性声明

本人声明所呈交的学位论文，是本人在导师指导下进行研究工作所取得的成果。除已特别加以标注和致谢的地方外，论文中不包含任何他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的贡献均已在论文中作了明确的说明。

作者签名：_____　　签字日期：_____

中国科学技术大学学位论文授权使用声明

作为申请学位的条件之一，学位论文著作权拥有者授权中国科学技术大学拥有学位论文的部分使用权，即：学校有权按有关规定向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅，可以将学位论文编入《中国学位论文全文数据库》等有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。本人提交的电子文档的内容和纸质论文的内容相一致。

控阅的学位论文在解除后也遵守此规定。

☑公开　□控阅（____ 年）

作者签名：_____　　导师签名：_____

签字日期：_____　　签字日期：_____

摘要

随着大规模 Transformer 模型参数规模和上下文长度的不断增长，注意力机制在推理过程中的计算与访存开销持续上升，性能瓶颈逐渐由计算开销转向访存开销。尤其在解码阶段，对历史键值缓存的持续访问使系统性能进一步受限于访存效率。近存计算作为一种能够缩短数据传输路径、提升数据局部性并缓解“存储墙”瓶颈的新型体系结构，为大模型高效推理提供了重要支撑。然而，面向近存计算节点内部的注意力机制推理优化，仍面临数据流组织复杂、软硬件参数强耦合等挑战，导致节点内部映射策略与体系结构配置难以实现协同优化。

面向近存计算架构的层次化数据流映射表示方法：为了对近存计算架构内部注意力机制推理过程进行结构化建模，本文提出了一种面向预填充阶段和解码两类典型阶段，并支持 MHA（多头注意力）、MQA（多查询注意力）和 GQA（分组查询注意力）等不同头部组织形式的层次化数据流映射表示方法。首首先，构建统一的注意力负载参数化模型，通过数据流图统一刻画张量依赖关系及不同阶段的计算与访存行为，为后续映射建模与性能分析提供统一抽象基础。在此基础上，从算子融合方式、张量数据放置方式和存储层级绑定的分块策略三个方面构建层次化映射树，对节点内部任务划分、数据驻留位置和执行顺序进行统一建模。该表示方法能够较完整地刻画节点内部的不同数据流模式，并有效剪枝庞大的映射空间。实验结果表明，本文提出的层次化数据流映射方法可将节点内部数据流映射空间缩减 91%，为后续设计空间探索提供基础。

面向近存计算架构的软硬件协同设计空间探索方法：为了在大规模高维参数空间中高效获得优质设计方案，本文提出了一种面向近存计算节点的软硬件协同设计空间探索方法。该方法综合考虑硬件配置参数、算子融合参数、数据放置参数和分块参数等多个维度，构建了面向节点内部注意力推理的软硬件协同设计空间。针对设计空间规模庞大且不可行候选解占比较高的问题，进一步提出约束感知的设计空间剪枝方法，从结构合法性、片上存储容量约束及带宽可满足性等方面对候选解进行过滤，从而有效缩减后续搜索范围。在此基础上，采用多目标序贯优化方法，对执行延迟、能耗及 DRAM 数据搬运量等关键指标进行联合建模与迭代搜索，实现了对 Pareto 最优解集的高效逼近。

此外，本文构建了由片上解析式性能模型与片外 Ramulator 详细时序仿真组成的协同评估框架，能够综合表征节点内部数据传输、阵列计算与片外存储访问行为，并为设计空间探索提供精确评估支撑。为面向大模型推理的近存计算体系结构优化提供了统一的方法基础与理论支持。

关键词：近存计算微架构，注意力推理，软硬件协同设计，设计空间探索，多目标优化

ABSTRACT

As large Transformer models continue to grow in both parameter scale and context length, 注意力 inference is increasingly constrained by high computational cost, intensive data movement, and limited memory bandwidth. This challenge becomes particularly severe in the decoding stage, where repeated accesses to historical key/value caches make performance highly dependent on memory efficiency. Processing-in-memory (PIM) offers a promising solution by shortening data movement paths, improving data locality, and mitigating the memory-wall bottleneck. However, optimizing 注意力 inference within a PIM node remains difficult due to the complexity of intra-node dataflow organization and the strong coupling between software and hardware design choices.

To address these challenges, this thesis proposes a hierarchical dataflow mapping representation for 注意力 inference within PIM nodes. The proposed method supports both prefilling and decoding stages, as well as multiple head organization schemes, including multi-head 注意力 (MHA), multi-query 注意力 (MQA), and grouped-query 注意力 (GQA). It first establishes a unified parameterized workload model and a dataflow graph to capture tensor dependencies and stage-specific execution characteristics. It then organizes intra-node task partitioning, data residency, and execution order by jointly modeling data placement, storage-hierarchy-aware tiling, and hierarchical execution trees. This structured representation accurately characterizes intra-node dataflow behaviors while substantially reducing the mapping parameter space.

Building on this representation, this thesis further develops a hardware-software co-design space exploration method for PIM nodes. The design space jointly considers hardware configuration, operator fusion, data placement, and tiling parameters. To improve search efficiency in this large high-dimensional space, a constraint-aware pruning strategy is introduced to eliminate infeasible candidates according to structural legality, on-chip storage capacity, and bandwidth feasibility. A multi-objective sequential optimization method is then applied to jointly optimize latency, energy consumption, and DRAM data movement, enabling efficient approximation of the Pareto frontier.

In addition, a collaborative evaluation framework is constructed by integrating an on-chip analytical model with detailed off-chip timing simulation based on Ramulator. The framework captures intra-node data transfer, array computation, and off-chip memory access behavior in a unified manner, providing accurate support for design space

exploration. Overall, the proposed methods establish a unified foundation for PIM architecture optimization toward efficient large-model inference.

KEY WORDS: PIM node microarchitecture, 注意力 inference, hardware-software co-design, design space exploration, multi-objective optimization

目录

第 1 章　绪论 1
1.1　研究背景与意义 1
1.1.1　深度学习与深度神经网络 1
1.1.2　神经网络推理加速 2
1.1.3　近存计算架构 6
1.1.4　神经网络的算子融合 8
1.2　国内外研究现状 9
1.2.1　近存计算架构 9
1.2.2　近存计算架构下神经网络推理加速 10
1.2.3　神经网络加速器设计空间探索算法 12
1.3　本文的研究内容与主要贡献 13
1.4　本文的组织架构 14
第 2 章　近存计算架构设计空间探索基础 15
2.1　近存计算架构 15
2.1.1　DRAM 近存计算架构 15
2.2　Attention 算子负载特征与参数空间 17
2.2.1　推理阶段的访存与计算特性 17
2.2.2　多种 Attention 变体 17
2.3　多目标设计空间探索理论基础 18
2.3.1　近存架构的优化目标 19
2.3.2　多目标优化问题基本概念 20
2.3.3　多目标优化算法基础 20
2.4　本章小结 25
第 3 章　近存架构 Attention 推理的层次化数据流映射方法 26
3.1　注意力推理负载表示方法 26
3.1.1　注意力计算的负载特征表示 26
3.1.2　Attention 负载编码框架 27
3.2　近存计算架构下注意力计算数据流映射方法 29
3.2.1　注意力算子分组融合方法 29
3.2.2　注意力算子 placement 模式表示方法 32
3.2.3　存储层级绑定的 Tiling 表示方法 34

3.2.4 层次化映射树的生成方法 35
3.3 本章小结 36
第 4 章 近存计算架构软硬件协同设计空间探索 37
4.1 近存计算架构设计特征表示 37
4.1.1 关键参数 37
4.1.2 近存计算节点的软硬件表征模型 39
4.2 近存计算节点软硬件协同设计空间探索 39
4.2.1 整体流程 39
4.2.2 设计空间建模 39
4.2.3 设计空间剪枝与初始化方法 41
4.2.4 层次化约束感知的设计空间探索框架 43
4.3 实验结果与分析 46
4.3.1 性能仿真平台 46
4.3.2 基于 Ramulator 的 DRAM 仿真 46
4.3.3 实验设置 47
4.3.4 消融实验 47
4.3.5 软硬件协同设计空间探索算法的效果 47
4.4 本章小结 48
参考文献 49
致谢 57
在读期间取得的科研成果 58

插图和附表清单

图 1.1 Transformer 架构图[6] 3
图 1.2 近年来 SOTA 深度学习模型参数规模演化[22] 3
图 1.3 算力、存储器带宽和互连带宽增长趋势图[22] 4
图 1.4 3D 堆叠式 DRAM 架构概述[47] 8

图 2.1 近存计算架构示意图[71] 16
图 2.2 近存计算节点微架构示意图 16
图 2.3 三种注意力计算数据流模式对比 18
图 2.4 帕累托前沿 21

图 3.1 注意力计算数据流 29
图 3.2 解码侧注意力算子分组融合计算示意图 31
图 3.3 典型数据 placement 模式 33

图 4.1 总体设计概览 40

表 1.1 主流处理器架构对比 6

表 4.1 单节点近存计算架构硬件设计参数 39
表 4.2 联合设计空间的规模 40
表 4.3 nDRAM 模型中可配置的 DRAM 参数及其含义 47

第 1 章　绪论

1.1　研究背景与意义

1.1.1　深度学习与深度神经网络

近年来，人工智能（Artificial Intelligence, AI）技术正经历从“判别式”向“生成式”的范式演进。借助于机器学习（Machine Learning, ML）算法在海量数据上的大规模预训练，计算机已实现从底层特征提取到高层语义理解的飞跃。作为机器学习的核心动力，深度学习通过构建多层神经网络（Neural Network, NN），利用非线性变换实现对复杂数据流形的端到端建模。随着计算算力的指数级增长与数据规模的爆发，深度神经架构已从早期的特定任务模型，逐步演变为具备通用处理能力的标准化架构[1]。

在计算机视觉（Computer Vision, CV）领域，卷积神经网络（Convolutional Neural Network, CNN）曾长期占据主导地位。其通过局部感受野与权值共享机制，模仿生物视觉系统的层次化特征感知，在图像分类[2]、目标检测[3]等任务中表现卓越。然而，近年来视觉大模型[4]（Vision Transformer, ViT）的兴起打破了这一格局，证明了基于全局注意力机制的架构在处理大规模视觉数据时具有更强的扩展性与表征上限。目前，CV 领域的研究重点正转向如何结合 CNN 的归纳偏置与 Transformer 的全局建模能力，以实现更高效的特征表达。

在自然语言处理（Natural Language Processing, NLP）领域，技术路线经历了从循环神经网络（RNN）到 Transformer 架构的彻底变革。传统的 RNN 及其变体 LSTM 由于循环递归的计算特性，在处理长程依赖时存在梯度消失与串行计算效率低下的瓶颈[5]。2017 年提出的 Transformer 架构如图1.1[6] 所示）通过自注意力（Self-Attention）机制实现了全序列并行计算，极大地增强了对全局上下文的捕获能力。这一架构不仅统一了 NLP 的技术路径，更通过跨模态改造，成为了计算机视觉、语音识别乃至多模态感知（Multimodal Perception）的通用骨干网络。

当前，以 GPT、Llama 为代表的生成式大语言模型（Large Language Model, LLM）标志着人工智能进入了“大模型”时代。通过增加参数规模、训练数据量与计算资源，模型涌现出了卓越的逻辑推理、代码生成及零样本学习能力[7-9]。进一步地，大模型正向着大字多模态模型（Large Multimodal Model, LMM）演进，能够同时理解并生成文本、图像、视频及音频，如 Gemini，GPT-4[10-11]等，为通向通用人工智能[12]（Artificial General Intelligence, AGI）提供了技术路径 [25]。

然而，尽管深度神经网络性能卓越，其庞大的参数量与复杂的算子逻辑也给

底层硬件带来了严峻的计算与存储挑战[13]。尤其是在长序列处理场景下，自注意力机制的计算复杂度随序列长度呈平方级增长，这使得模型在边缘侧设备上的部署面临严重的功耗与时延瓶颈[14-15]。因此，研究高效的神经网络加速技术与新型计算架构，已成为当前学术界与工业界共同关注的前沿热点[16]。

在自然语言处理及跨模态学习领域，基于注意力机制的 Transformer 架构已成为当前的主流模型方案。相比于传统的循环神经网络，Transformer 通过自注意力（Self-Attention）机制实现了全序列的并行计算，能够有效捕捉全局依赖关系。然而，随着应用场景对序列长度 N 的需求不断增加，Transformer 的核心算子 Self-Attention 在计算与存储开销方面呈现出显著的可扩展性瓶颈，进而限制了其在长序列场景下的高效应用[17-18]。

具体而言，给定输入序列 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$，其中 N 为序列长度，d 为注意力头的维度。Attention 的标准计算流程需要通过查询矩阵（Query）与键矩阵（Key）的交互得到注意力得分，再作用于值矩阵（Value）以实现特征聚合。其数学表达如式（3.1） 所示[6]：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^{\top} \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d} \tag{1.1}$$

从数据流的角度分析，该计算过程产生的中间矩阵 $\mathbf{S}$ 与 $\mathbf{P}$ 的维度均为 $N \times N$，这意味着 Self-Attention 在序列长度为 N 时具有 $O(N^2)$ 的时间和空间复杂度。这种平方级的增长特征，使得模型在处理超长文本或高分辨率图像等长序列场景[19]时，面临严重的运行时间延迟与显存压力。

如图 1.1 所示，Transformer 架构虽在特征表征上展现了卓越性能，但其底层算子在执行过程中存在严重的访存开销。在现代计算架构中，由于处理器算力（如 Tensor Cores）[20]与内存带宽（HBM）之间的增长不匹配，这种 $O(N^2)$ 的计算模式往往受限于“内存墙”瓶颈[21-23]，导致算力资源无法得到充分利用。所谓“内存墙”（memory wall），是指处理器计算性能的提升速度长期快于存储系统（DRAM/HBM）带宽与延迟的改善速度，使得越来越多的程序在端到端执行时间与能耗上由数据搬运而非算术运算主导。

1.1.2　神经网络推理加速

随着深度学习凭借卓越的性能迅速渗透各行各业，其推理阶段的计算性能及能效问题已成为制约大规模应用的核心瓶颈。从早期的卷积神经网络（CNN）到现代的大语言模型（LLM），模型参数规模呈现出指数级增长。如图 1.2所示，模型参数量在短短数年内从亿级飙升至千亿乃至万亿级别。对于 Transformer 这一当前主流的架构而言，推理加速的难度不仅在于庞大的计算量，更在于其独特的访存特征。随着深度学习模型规模与序列长度增长，这一矛盾在注意力等高访存算子中尤为突出：即使计算单元具备极高峰值吞吐，仍可能因带宽受限而难以

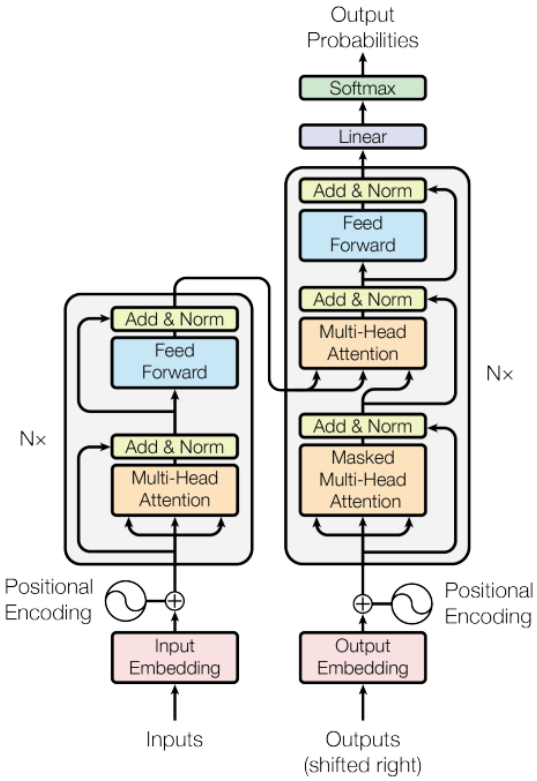


图 1.1　Transformer 架构图[6]

达到理想利用率。如图如图 1.3展示了存储带宽提升滞后于算力增长，因此，如何优化自注意力机制的数据流，降低其计算复杂度与访存频率，已成为当前深度学习系统优化领域的核心课题之一。

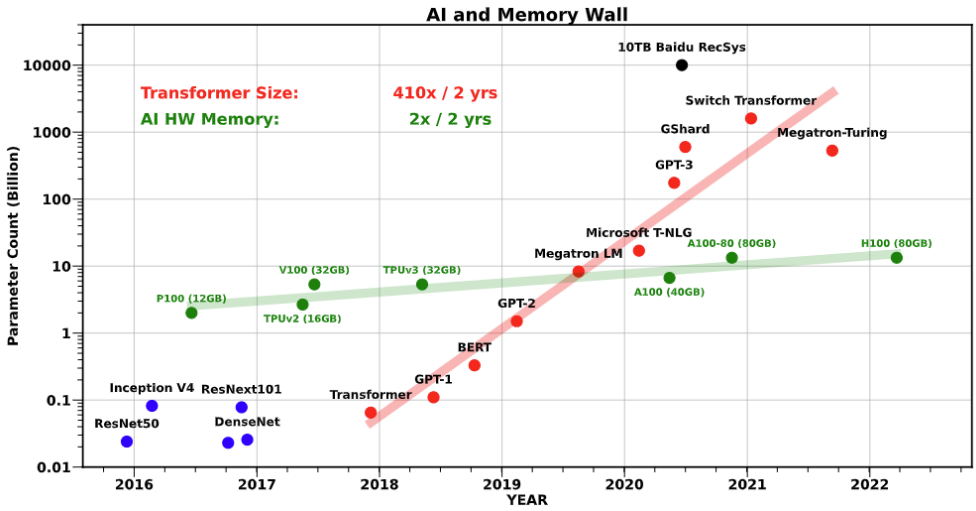


图 1.2　近年来 SOTA 深度学习模型参数规模演化[22]

在推理过程中，Transformer 与 CNN 表现出截然不同的计算特性。CNN 的核心在于卷积算子，具有显著的局部感知和权值共享特征[24]，易于通过片内缓存实现高度的数据重用[25-26]。相比之下，Transformer 的核心计算单元为多头自注

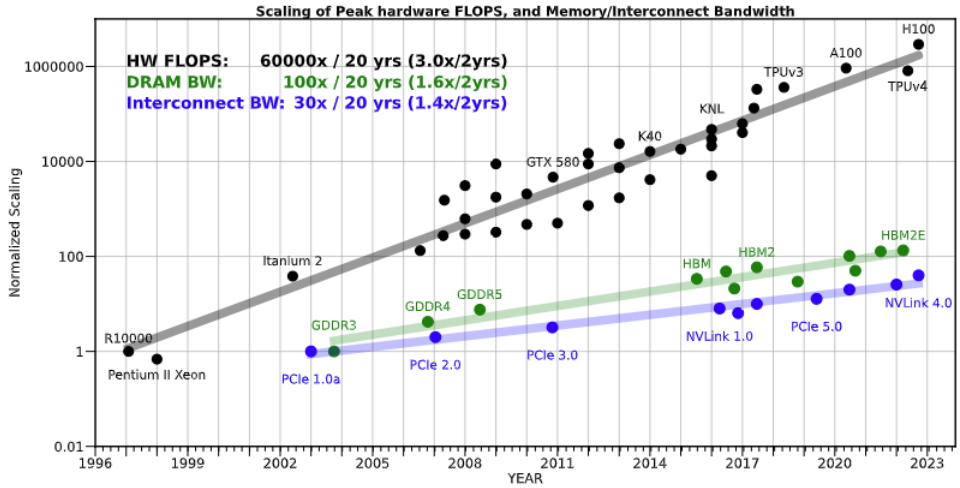


图 1.3　算力、存储器带宽和互连带宽增长趋势图[22]

意力机制（Multi-Head Self-Attention, MHSA）。在自回归生成任务中，其推理流程通常可划分为两个阶段：**预填充（Prefill）阶段**与**解码（Decoding）阶段**[27]。

- **预填充（Prefill）阶段：**模型对输入提示（prompt）进行一次性前向计算，得到该序列对应的注意力键（Key）与值（Value）表示，并将其写入缓存，用于后续生成过程中的注意力计算。
- **解码（Decoding）阶段：**模型以自回归方式逐 token 生成输出。为避免对历史 token 反复计算，模型需要持续保存并更新已生成 token 的 Key/Value 表示，即 **KV Cache**。随着序列长度不断增长，KV Cache 的容量开销与读写带宽需求迅速上升，使得解码阶段的推理逐渐呈现出显著的**访存密集型（memory-bound）**特征，从而成为时延与能效的主要限制因素。

针对上述挑战，目前的推理加速技术正从算法与硬件两个维度并行演进。在算法与软件层面，针对显存墙问题，学术界提出了多项优化技术：以 FlashAttention 为代表的算子融合技术，通过重构自注意力机制的数据流[28]，利用 SRAM 进行分块计算，显著减少了对片外存储（HBM）的访问频率；PagedAttention 等显存管理技术引入了操作系统中虚拟内存的分页思想[27]，大幅优化了 KV Cache 的存储碎片问题。此外，低位宽量化（如 FP8、INT4）[29]与推测性解码[30]（Speculative Decoding）也成为了在有限带宽下提升吞吐率的关键手段。

在硬件平台层面，为了应对差异化的计算需求，多种硬件架构应运而生，并在深度学习推理任务中展现出各自独特的优势与局限性。

通用处理器（CPU）：通用处理器（CPU）：CPU 通常采用以冯・诺依曼模型为基础的通用指令执行架构，内部集成分支预测、乱序执行等复杂控制逻辑，并配备多级缓存层次结构以降低访存延迟，从而在控制流复杂或串行依赖性强的任务中具备较高的通用性与灵活性。然而，相比面向数据并行的加速器，CPU 的计算资源（如向量/矩阵运算单元）在芯片资源占比与并行组织形式上更偏向

通用执行与控制开销，使其在神经网络推理中以大规模矩阵乘法为主的计算模式下，往往难以充分发挥峰值吞吐并获得理想能效。针对典型矩阵乘法工作负载的实测研究也表明，相比离散 GPU，通用 CPU 在性能与能效上通常存在显著差距。[31] 此外，CPU 的外部内存带宽与层次化缓存容量在长上下文大模型推理等场景下更易成为瓶颈，难以满足实时推理对高吞吐与低时延的需求。

图形处理器（GPU）：GPU 最初面向图形渲染与吞吐密集型并行计算设计，通常采用单指令多线程（SIMT）执行模型，通过大量线程并行与“延迟隐藏”机制提升对数据并行工作负载的处理效率。其编程模型（如 CUDA）将并行任务组织为线程块与线程网格，从而便于在大规模并行场景下实现高吞吐计算[32]。随着深度学习的兴起，现代数据中心 GPU（以 NVIDIA H100 为代表）进一步引入专用的 Tensor Core 单元与 Transformer Engine，对矩阵乘加（GEMM）与 Transformer 相关算子提供硬件级加速，并在存储子系统侧配备高带宽显存（如 HBM3/HBM3e）以提高带宽供给、缓解数据搬移瓶颈。因此，结合 CUDA 及其深度学习基础库（如 cuDNN）所形成的成熟软件生态，GPU 仍是当前大语言模型训练与推理的主流平台之一。然而，GPU 在提供极高峰值算力的同时通常伴随较大的板级功耗与散热压力（例如 H100 不同形态具有数百瓦至 700W 级别的可配置 TDP），在部署与运维层面带来显著成本。另一方面，GPU 的通用存储层次与缓存/内存访问机制主要面向致密规则计算而优化：当模型引入稀疏化或低位宽量化时[33]，若稀疏结构与硬件支持（如特定稀疏模式与内核实现）不匹配，或量化算子/数据布局导致访存与指令效率受限，可能出现计算单元利用率不足与能效浪费，进而使性能提升受限。

现场可编程门阵列（FPGA）[34]：FPGA 常被视为“准专用”计算平台，其片上集成大量可编程逻辑资源（如查找表 LUT、触发器）以及面向乘加运算的数字信号处理单元（DSP）和片上存储（BRAM/URAM）。与 CPU/GPU 采用固定微架构不同，FPGA 允许开发者依据特定算法的计算图与数据流特性对硬件结构进行重构，将算子映射为空间并行与深流水的数据流架构，从而在流式推理场景中以较低时延、较高能效执行计算密集算子，并通过片上缓冲与数据复用减少片外访存开销[35]。在 Transformer 等模型快速迭代的背景下，FPGA 的可重构性使其更易支持非标准数值格式与结构化/非结构化稀疏等定制化计算模式；已有工作对 FPGA/ASIC 的 Transformer 推理设计、压缩与结构优化进行了系统梳理，并指出其在功耗受限或对时延敏感的特定应用中具有潜在优势。[34] 但需要注意的是，FPGA 通常工作频率低于高端 GPU/CPU，且开发与部署需要硬件描述语言（HDL）或高层次综合（HLS）等工具链支撑，设计空间探索与时序收敛成本较高；因此，在追求通用性与快速迭代的大规模在线推理服务中，其应用门槛与工程复杂度仍然是限制因素。[36]

领域专用架构（DSA/ASIC）[34]：为突破通用处理器在能效与数据搬移方面的瓶颈，面向深度神经网络工作负载定制的领域专用加速器（Domain-Specific Accelerator, DSA/ASIC），如 Google TPU 与华为 Ascend 等，已成为学术界与工业界的重要研究方向[37-38]。此类架构通常弱化通用 CPU/GPU 中面向复杂控制流的冗余控制逻辑，将芯片资源集中用于规则、可并行的张量计算，并常以**脉动阵列（Systolic Array）**或类似的数据流矩阵引擎作为核心计算单元[39]，通过对片上数据流动路径与存储层次（片上 SRAM/片外 HBM/DDR）进行协同设计，提升数据重用率并降低片外访存，从而在训练与推理场景中获得更高的**性能/功耗比**。

表 1.1 详细对比了上述主流硬件平台的技术特性与应用场景。

表 1.1　主流处理器架构对比

类别	CPU	GPU	FPGA	DSA	ASIC
概述	通用处理；复杂控制；低并行	SIMT 并行；高吞吐；矩阵加速	可重构逻辑；数据流；现场更新	领域定制；张量引擎；数据流优化	应用定制；硬连线；不可修改
处理能力	低/中并行；强控制；分支友好	大规模并行；高 FLOPS；高带宽	空间并行；深流水；片上复用	专用算子；高效并行；高利用率	极致优化；低开销；最高能效
编程方式	C/C++；Python；OS/线程	CUDA；OpenCL；kernel 编程	HDL；HLS；时序收敛	专用 ISA；编译器栈；算子库	固化逻辑；配置受限；不可编程
优势	生态成熟；易开发；低成本	训练/推理；吞吐强；生态完善	低时延；高能效；位宽灵活	能效优；算子融合；可控延迟	最佳能效；最低延迟；面积优化
劣势	吞吐受限；能效较低；内存墙	功耗高；成本高；稀疏/量化受限	开发复杂；频率较低；工具链重	通用性弱；迁移困难；生态依赖	NRE 高；迭代慢；灵活性差

尽管加速器性能不断提升，但在传统的冯·诺依曼架构下，计算单元与存储单元分离的设计始终难以从根本上解决“存储墙”问题。由于存储带宽的提升速度远滞后于算力的增长，在 LLM 推理场景下，数据搬运产生的能耗往往占据系统总能耗的 90% 以上。这种严重的算力与带宽失衡，迫使研究重心开始向“以存储为中心”的新型架构（如近存计算）转移。

1.1.3　近存计算架构

随着人工智能、大数据以及云计算等技术的快速发展，数据密集型应用对计算系统性能提出了更高的要求。以 Chat-GPT 为代表的大语言模型（Large Language Models, LLMs）的应用落地，对底层硬件算力提出了更高要求。大模型训练及推理任务具有参数规模大、数据访问频繁的特征，此外，不断涌现的新型算法，例如语音识别、图像处理及类脑计算，处理数据规模不断扩大，计算任务从计算密集型向数据（访存）密集型转变。数据密集型的计算任务具有两个特征：

数据集较大、数据局部性较差，即具有离散且随机的访存行为特征，这就需要在处理器和存储器之间进行频繁的数据传输，导致数据迁移消耗的能量开销远大于计算单元的运算操作。然而，在传统的冯·诺依曼体系结构中，计算单元与存储单元在物理上相互分离，二者之间有限的数据传输带宽难以满足处理器对数据吞吐量的需求，导致计算单元在执行过程中频繁等待数据访问，从而显著降低整体系统效率。此外，大规模数据在处理器与片外存储之间频繁搬运，其能耗往往远高于计算本身的能耗，这进一步加剧了系统的能耗问题。上述问题通常被称为“存储墙”。在许多数据密集型应用场景中，由于缓存数据难以被高效复用，大量数据需要在处理器与外部存储之间反复传输，使得“存储墙”问题更加突出。

在此背景下，计算架构的革新成为突破算力瓶颈的重要方向。存算一体[40-41]（Processing-in-Memory，PIM）技术通过将部分计算逻辑集成到存储器内部或其邻近区域，从而减少数据在计算单元与存储单元之间的频繁传输，显著降低数据移动带来的延迟与能耗。根据计算逻辑与存储阵列之间的集成方式不同，近存计算架构通常可以分为两类：近存计算（Processing-Near-Memory，PNM）和存内计算（Processing-Using-Memory，PUM）[42]。

近年来，随着三维堆叠技术的不断发展，存算融合架构的研究逐渐受到广泛关注。该技术通过在垂直方向集成多层芯片，缓解了传统平面工艺下逻辑单元与存储单元难以兼顾的问题。与二维集成方式相比，三维堆叠结构[43]能够将存储层与逻辑层分别采用不同工艺进行优化设计，使存储单元满足高密度集成需求的同时，逻辑层也能够兼顾计算性能，从而减少单一工艺折中带来的限制。基于这一思路，混合内存立方体[44]（Hybrid Memory Cube，混合内存立方体，HMC）、高带宽内存[45]（High Bandwidth Memory，高带宽内存，HBM）以及异质集成嵌入式动态随机存储器[46]（Stacked Embedded DRAM，堆叠式嵌入式动态随机存储器，SEDRAM）等三维堆叠存储结构相继发展起来。这类架构通常借助混合键合（Hybrid Bonding，混合键合，HB）或硅通孔（Through-Silicon Via，硅通孔，TSV）等互连方式，实现存储层与逻辑层之间的紧密集成，并在逻辑层中进一步引入计算功能。由此，不仅提高了芯片内部的数据访问效率，也为构建高带宽、低数据搬运开销的存算融合系统提供了重要支撑。

如图1.4所示，展示了基于 DRAM 的近存计算架构示意图。逻辑（计算）功能和存储功能分别放置在 3D 堆叠芯片的不同层上。逻辑层与 DRAM 层通过混合键合工艺集成。DRAM 的每个 bank 都有独立的高速数据通路与逻辑层连接，使得整个芯片具有很高的 DRAM 带宽。由于逻辑层与 DRAM 层之间的通信距离很短，近存计算芯片的访存能耗比传统的片外 DRAM 低 1 到 2 个数量级。基于 3D 堆叠存储的近存计算架构具有高带宽和低能耗的特点，因此正逐渐被应用于处理访存密集型应用。

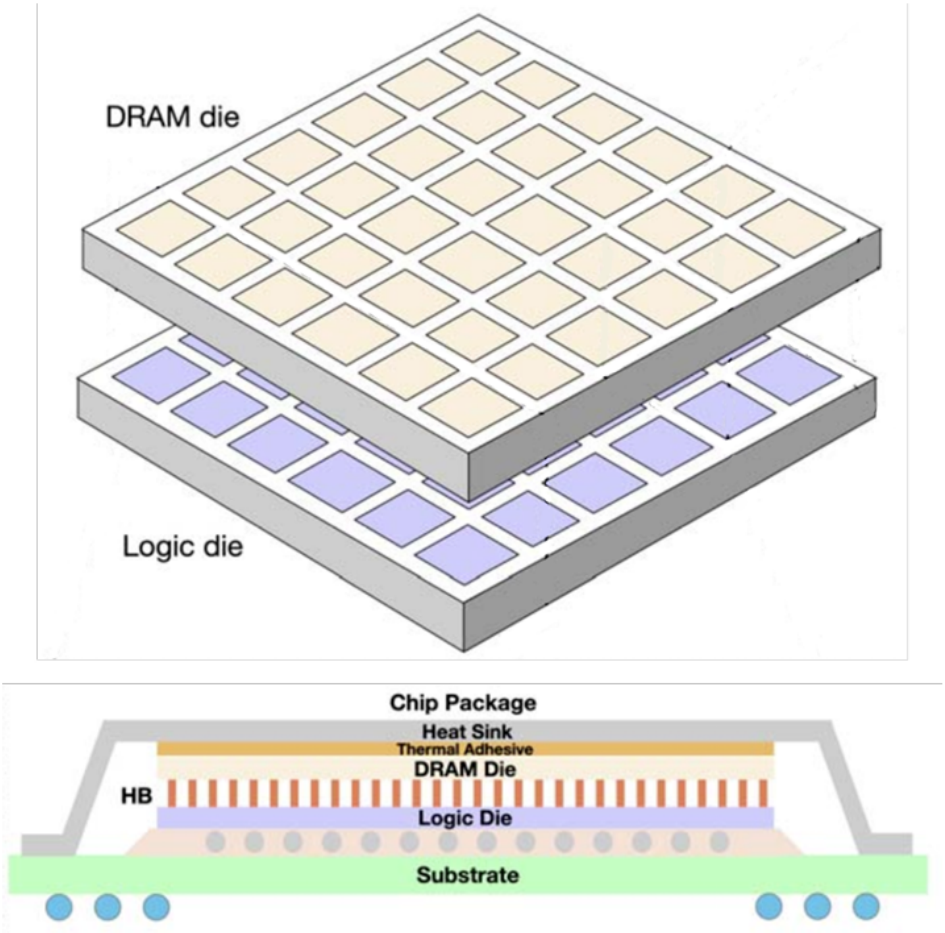


图 1.4　3D 堆叠式 DRAM 架构概述[47]

1.1.4　神经网络的算子融合

循环融合（Loop Fusion）是编译器优化中的一项重要技术，其核心思想是将多个具有相同或兼容迭代空间的循环合并为单一循环，从而提高数据局部性并减少中间结果的存储开销。通过循环融合，前一计算阶段产生的数据（Producer）可以在同一迭代中被后续计算阶段（Consumer）立即使用，使数据能够驻留在寄存器或高速缓存中，而无需写回主存，从而显著降低内存访问开销并提升程序性能[48-49]。

以简单的向量运算为例，对于两个依次执行的操作 $C = A + B$ 和 $E = C \times D$，若不进行优化，编译器通常会生成两个独立的循环。通过循环融合，可以将两个循环合并为一个单循环，在同一次迭代中完成加法和乘法计算，从而避免中间变量 C 被写入和再次读取主存。然而，在深度学习工作负载中，算子的计算模式往往更加复杂，使得循环融合面临新的挑战。

例如，当算子 A 包含归约操作时，如计算张量每一行的和，传统编译器通常需要在完成整行的归约计算后才能继续执行后续操作。若后续算子 B 需要利用该归约结果进行进一步计算（例如 Softmax 中的归一化过程），则会形成严格的

数据依赖关系。在典型的 GPU 执行模型中，这种依赖通常通过多个内核（Kernel）顺序执行来实现：首先执行算子 A 的内核，并通过全局同步确保结果写回全局内存；随后启动算子 B 的内核读取这些结果继续计算[50]。这种执行方式不仅引入额外的内核启动开销，还会破坏计算流水线，使得不同算子之间难以实现跨行或跨批次的并行处理。理想情况下，当算子 A 计算第 $i+1$ 行数据时，算子 B 可以同时处理第 i 行数据，从而形成流水线并行。然而，由于归约操作需要在完整数据范围内进行累积，其结果在计算完成前无法被安全使用，这使得传统编译器难以直接进行算子融合。如果强制进行融合，中间结果往往无法完全存放在寄存器中，导致寄存器压力显著增加。当寄存器资源不足时，编译器会将部分中间变量溢出到本地内存（Register Spilling），从而引入额外的内存访问开销，反而降低整体执行性能[51]。因此，如何在保证数据依赖正确性的前提下，实现复杂算子的高效融合与流水线执行，成为深度学习编译器与体系结构优化中的一个重要研究问题。

1.2　国内外研究现状

1.2.1　近存计算架构

近年来，随着深度神经网络（Deep Neural Network, DNN）模型规模持续增长以及推理阶段对权重与激活访问需求的急剧上升，传统冯・诺依曼体系结构中“计算–存储分离”所引发的数据搬运开销愈发突出，系统性能与能效逐渐受制于“内存墙”。为缓解这一瓶颈，学术界提出近存计算（Processing-in-Memory, PIM）/存内计算（In-Memory Computing, IMC）范式，通过将部分计算能力嵌入存储阵列或下沉到近存侧执行，以缩短数据路径、提升有效带宽利用率，从而在神经网络推理加速方面展现出显著潜力[52]。依托 3D 堆叠存储与新型存储器件的工艺演进，近存计算架构逐步从早期的概念验证发展到面向端到端推理的体系化设计，当前近存计算架构的研究主要集中在基于 SRAM、ROM 以及两者混合的异构架构设计上[53]。

静态随机存取存储器（SRAM）凭借其成熟的 CMOS 工艺兼容性、高读写速度及鲁棒性，是目前存算一体研究的主流介质。现有的 SRAM-CiM 研究涵盖了从电荷域、电流域的模拟计算到全数字域的高精度逻辑运算。然而，SRAM 单元（通常为 6T 或 8T）较大的面积开销导致其存储密度较低，且存在显著的静态漏功耗问题。这使得单纯基于 SRAM 的存算架构难以满足大模型在边缘端部署时对海量片上权重存储的需求，限制了其在存储受限场景下的进一步扩展。

为了解决存储密度与静态功耗的挑战，只读存储器（ROM）作为一种高密度的存储介质重新受到学术界的关注。ROM 利用单晶体管即可存储多比特信息

（Multi-level Cell），具有极高的面积效率和极低的待机功耗。相关研究表明，在权重固定的推理任务中，基于 ROM 的存算架构能够提供远超 SRAM 的能效比与存储密度，有效消除了片外权重加载的带宽瓶颈。然而，ROM 的非易失性和不可重写特性导致其缺乏灵活性，无法支持模型的在线微调或任务迁移，难以适应需要动态更新权重的自适应计算场景。

鉴于单一存储介质在密度与灵活性上的权衡困境，基于成熟 CMOS 工艺的 ROM-SRAM 异构混合存算架构（Hybrid CiM）成为当前的前沿研究热点。该架构通过“权重与结构自适应”的设计策略，利用高密度的 ROM 存储神经网络中绝大部分固定不变的权重参数，同时利用高灵活性的 SRAM 处理少部分需要微调的权重、激活值及动态中间结果[53]。在电路实现层面，研究者们正致力于解决混合架构中不同介质间的信号域对齐（如时域与数字域接口）及量化协同问题。此外，针对大模型在边缘端的落地需求，结合 3D 堆叠（3D Stacking）与芯粒（Chiplet）集成的扩展技术也正逐步应用于混合存算系统，通过先进封装技术垂直互联高密度 ROM 芯粒与高性能 SRAM 逻辑芯粒，以打破单芯片的容量与算力限制。

1.2.2　近存计算架构下神经网络推理加速

在深度学习推理任务中，大量计算由张量运算和数据搬运组成，其性能往往受到内存带宽和数据移动开销的限制。为系统地评估不同架构设计与数据流映射策略对性能和能效的影响，研究者提出了一系列面向深度学习加速器的建模与优化框架。2019 年，英伟达及麻省理工学院的 Parashar 等人提出了 Timeloop 建模框架[54]，以解决加速器设计空间评估缺乏系统化工具的问题。Timeloop 通过统一且简洁的架构表示方法，能够描述广泛的硬件拓扑结构。其高性能分析模型和自动映射器（Mapper）在庞大的映射空间中自动搜索最优的循环分块（Tiling）、调度以及数据驻留策略；分析模型则利用神经网络工作负载的规律性，对性能和能效指标进行解析式预测。该框架填补了架构设计空间与映射策略之间耦合关系建模的空白，使研究者能够在统一平台上对不同硬件架构进行公平比较，并揭示数据流与存储层次协同设计在提升系统效率中的关键作用。

随着深度学习模型结构的复杂化，算子融合（Operator Fusion）逐渐成为提高推理性能的重要手段。然而，传统性能建模方法通常针对单算子优化，难以准确刻画多个算子融合时的资源竞争和中间数据驻留特征。2023 年，北京大学 Zheng 等人提出 TileFlow 建模框架[55]，用于分析算子融合场景下的数据流设计空间。TileFlow 将融合数据流的设计空间定义为由计算顺序、资源绑定和循环分块构成的三维空间，并引入以分块为核心的符号表示系统。通过基于分析树的建模方法，该框架能够自下而上地统一计算数据搬运量和资源利用率，并结合

遗传算法与蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）在复杂设计空间中高效寻找最优映射策略。实验结果表明，在自注意力机制和卷积链等任务中，TileFlow 发现的数据流方案相比传统方案可分别实现平均 1.85 倍和 1.28 倍的性能提升。

在编译器层面，为进一步减少数据搬运并提升算子执行效率，现代深度学习编译框架通常采用灵活的中间表示（Intermediate Representation, IR）来支持复杂算子的融合与调度优化。例如 Halide、TVM 和 Triton 等编译器提出了“算法与调度分离”的设计思想[56-58]。在这些框架中，算法用于描述计算逻辑，即“算什么”，而调度用于描述计算执行方式，即“怎么算”。这种设计使得开发者或自动调度器能够通过不同的调度策略探索性能优化空间，从而实现算子融合、循环变换以及数据局部性优化。

除了编译器层面的优化，近年来也出现了一些面向深度学习的数据流加速架构。例如，SambaNova[59]提出的可重构数据流单元采用空间计算模式，将不同算子映射到芯片上的不同计算区域。其 SN40L RDU 芯片由大量 Pattern Compute Unit（PCU）和 Pattern Memory Unit（PMU）组成，并通过片上交换网络互联。在编译阶段，SambaFlow 编译器会根据算子的计算需求与内存需求进行空间划分，将不同算子映射到不同的 PCU 区域，并通过交换网络建立专用的数据流通路。

近年来，Transformer 与大语言模型（Large Language Model, LLM）推理快速发展，使研究进一步从算子级加速走向系统级异构协同。由于 LLM 推理中既包含计算密集的矩阵乘（GEMM）阶段，也包含强带宽敏感的矩阵–向量乘（GEMV）与缓存访问（如注意力相关访存）阶段，单一计算范式难以兼顾端到端效率。由此，NPU/GPU 与 PIM 的异构融合成为重要趋势：通过将计算密集型算子分配给 NPU/GPU，将带宽受限的 GEMV/访存型算子映射至 PIM 执行，从而发挥“算力型单元 + 带宽型单元”的互补优势。2024 年，NeuPIMs 提出 NPU 与 PIM 协同的异构推理架构，通过任务划分与调度将 LLM 推理中的关键带宽瓶颈环节下沉至 PIM，以提升批量推理吞吐[60]。进一步地，面向端到端推理的数据共享与通信开销问题，相关研究开始引入统一内存视角，通过构建 NPU–PIM 统一内存系统并配合访问调度机制减少数据复制与跨域通信，从而在大规模模型推理中获得更稳定的系统级收益。

随着近存计算架构规模扩大与参数组合增多，设计复杂度迅速上升，传统依赖专家经验的手工设计难以高效探索最优解。为此，研究逐步强调方法学与工具链建设：一方面，通过抽象统一的计算/存储资源模型，将神经网络计算图转换为可映射的执行流程；另一方面，引入设计空间搜索与映射优化，在面积、功耗、吞吐与精度约束下自动生成架构配置与数据流策略。2024 年，PIMSYN[61]通过自动化综合与探索机制实现 CNN 到 PIM 加速器的快速生成与优化，降低设计门

槛并提升设计效率。

总体来看，近存计算架构已成为突破神经网络推理内存瓶颈的重要技术路线：从 3D 堆叠 DRAM 近存计算到新型存储器件的存内计算阵列，再到面向 LLM 推理的异构系统协同与统一内存机制，相关研究在性能与能效方面均取得了显著进展。然而，该领域仍面临若干关键挑战：其一，模拟/混合信号存内计算的误差建模与精度保障仍需进一步完善；其二，异构系统的编程模型、任务划分与调度机制尚缺乏统一范式；其三，统一内存管理与一致性支持会引入新的系统复杂性；其四，大规模模型在 PIM 上的映射优化与端到端评测方法仍有待标准化。未来研究亟需在体系结构、编译映射与系统软件协同等层面进一步融合，以实现面向复杂神经网络尤其是 LLM 推理的高效、可扩展近存计算加速。

1.2.3　神经网络加速器设计空间探索算法

设计空间探索作为神经网络加速器自动化设计中的关键环节，能够在面对海量硬件候选方案、映射策略组合以及多目标约束（性能、能耗、面积与带宽等）的复杂挑战时，快速挖掘最具潜力的体系结构与映射参数配置。随着网络规模增长、算子形态多样化以及存储层级愈发复杂，传统依赖经验的手工调参与穷举式搜索难以满足设计效率要求。由此，围绕“设计空间参数化建模—代价模型构建—搜索算法设计—软硬件协同落地”的数据驱动与模型驱动融合探索方法，逐渐成为国内外研究的主流方向。

2017 年，研究者提出将多目标贝叶斯优化引入加速器设计空间探索的范式[62]。该类方法面向“评估代价高、空间维度高、目标冲突明显”的典型 DSE 场景，利用代理模型刻画设计点与目标指标之间的关系，并通过采集函数在探索与利用之间动态权衡，从而在有限评估预算下逼近 Pareto 前沿。该工作奠定了后续以 Bayesian optimization 为核心的自动化 DSE 方法论基础，并推动了“以少量仿真换取高质量设计点集”的研究路线。

2019 年，Timeloop 框架系统化地统一刻画了硬件拓扑、存储层级与数据流/映射选择，形成可组合、可搜索的设计空间表达，并提供配套的映射搜索流程以获得近优映射方案[54]。与之互补，MAESTRO 提出了以数据复用与数据流为中心的分析型代价模型，通过输入网络层描述、硬件资源约束与数据流参数，快速输出性能与能耗等估计结果，从而为大规模 DSE 提供可扩展的目标函数评估能力[63]。上述两类代表性工作分别从“空间如何定义”与“候选点如何评估”两个维度，为后续自动化探索框架提供了基础设施与可对比基线。

2020 年以后，针对映射空间爆炸与硬件资源分配耦合增强的问题，研究者开始围绕“映射编码—搜索策略—资源配置”的联合优化提出更具针对性的算法体系。Interstellar 以调度/循环变换为核心形式化工具，将加速器微结构与映射

决策纳入统一表达，强调通过程序变换视角对数据流与存储访问进行系统建模，从而提升设计空间参数化的可解释性与可迁移性[64]。面向映射搜索的高维离散特性，GAMMA 通过面向映射问题定制遗传编码与变异交叉策略，将庞大的映射组合压缩为可高效进化的搜索过程，在保证可行性的同时提升收敛效率[65]。另一方面，ConfuciuX 聚焦于给定数据流风格下的硬件资源分配问题，引入强化学习进行粗粒度决策，并结合进化式微调实现对 PE 阵列与片上缓冲等关键资源的自动分配，从而在性能与能效约束下实现更稳定的硬件配置搜索[66]。同一时期，AutoDNNchip 将性能/能耗预测器与设计空间探索流程进一步与自动化实现闭环结合，面向 FPGA/ASIC 等不同落地平台构建端到端的“预测—搜索—生成”工具链，体现出 DSE 方法从算法验证向工程可用性演进的趋势[67]。

在框架化与可复用性方面，除 Timeloop/MAESTRO 路线外，ZigZag 等开源 DSE 工具强调硬件—映射协同探索与快速代价估计，通过模块化建模与统一接口支持对不同存储层级与映射策略的组合评估，逐步形成与主流框架并行的工具生态[68]。这类工作通常将“硬件资源定义、映射合法性约束、代价评估与搜索算法”解耦，使得研究者能够在统一平台上对不同搜索策略与建模假设进行可重复对比，从而降低方法迁移与复现实验的门槛。

2022 年，TRIM 从更系统的 EDA/模型化视角出发，将加速器效率分解为映射策略、网络层特征与硬件参数三类因素的耦合结果，并据此构建更具泛化能力的探索模型，以支持跨网络、跨硬件配置的效率预测与设计点筛选[69]。该方向强调以统一表征与结构化特征建模提升代价模型的稳健性，为后续跨场景 DSE 提供更可迁移的评估与搜索基础。进一步地，CSDSE 引入多智能体协作与多尺度搜索机制，通过分工化的探索/利用策略与层次化搜索流程，在复杂多目标约束下提高搜索效率与解质量，体现出 DSE 搜索算法向智能化、自适应与协同决策演进的趋势[70]。

神经网络加速器设计空间探索研究已从早期以启发式采样与单一目标优化为主，逐步发展为以“统一空间表述 + 快速代价模型 + 多目标智能搜索 + 端到端工具链落地”为核心的系统化技术体系。未来研究一方面将继续强化跨模型/跨平台的代价模型泛化能力与可解释性，另一方面也将推动搜索策略与编译映射、资源分配、实现约束的更深度耦合，以进一步提升自动化设计效率与 Pareto 优化质量。

1.3　本文的研究内容与主要贡献

不考虑近存 PIM 阵列节点之间的数据传输，而是规定一个静态任务给一个 PIM 节点，考虑节点内部微架构的数据流划分策略以及微架构参数的联合设计

空间探索。

1.4　本文的组织架构

第 2 章　近存计算架构设计空间探索基础

硬件设计空间探索（Design Space Exploration, DSE）是指在给定的性能、功耗等物理约束条件下，通过系统化方法对硬件架构参数与映射策略进行组合评估与优化，以寻找最优或近似最优设计方案的过程。硬件设计空间探索本质上是一个高度复杂的多目标优化问题（Multi-Objective Optimization, MOO），其核心挑战在于实现多个相互制约的性能指标（如执行周期、能耗、面积开销）之间的最优权衡[54]。在近存（Processing-in-Memory, PIM）架构应用于 Transformer 模型推理加速的场景中，如何平衡局部计算单元的利用率与全局 DRAM 访存带宽，尤为关键。

2.1　近存计算架构

2.1.1　DRAM 近存计算架构

由于 DRAM 的存储密度更高、单位容量成本更低，因此能够在有限芯片面积和成本约束下提供更大的数据存储能力，本研究采用基于 DRAM 的近存计算架构。图 2.1直观地展示了该架构的整体设计概况。架构在物理层面上由面积相等的堆叠存储芯片与逻辑芯片构成，利用硅通孔（TSV）等先进的 3D 堆叠技术，在保持常规 DRAM 同等容量的前提下，实现了访存带宽的大幅跃升。逻辑层被划分为多个计算阵列，其中一个子阵列与底层的若干独立并行 DRAM Bank 共同组成一个完整的近存计算节点（PIM-Node）。各节点采用二维网格拓扑进行组织，通过逻辑层内部的片上网络（NoC）与路由器（Router）实现横向及纵向的高效通信，构建出具备高扩展性的互连结构。这种近数据处理模式极大地减少了数据在片外的长距离搬运，显著降低了系统的整体能耗。

为满足 Transformer 推理任务中高并发访存与非线性计算的严苛需求，本研究对单个 PIM 节点的微架构进行了参数化建模。如图 2.2所示，单个 PIM 节点由存储子系统、数据引擎（Data Engine, DE）与计算子系统三部分协同构成的层次化硬件框架。各子模块紧密配合，共同完成高效的数据存取与计算执行：

存储子系统作为 PIM 架构的底层数据支撑，其核心设计目标在于充分释放 DRAM 的内部高带宽优势并提供灵活的访问并行度。DRAM 作为节点的主存储介质，其带宽指标直接决定了系统的数据供给能力。随着 DRAM 带宽的提升，计算子系统能够获得更高的峰值数据供给速率，从而有效缓解访存阶段对整体执行效率的制约。

数据引擎枢纽式地桥接于存储与计算子系统之间，其内部集成了 SRAM 作

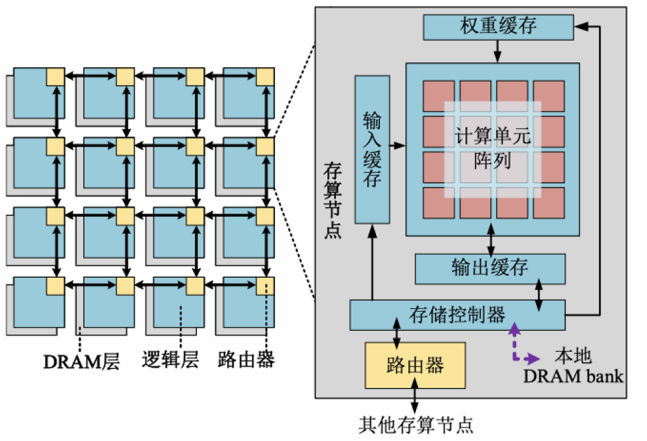


图 2.1　近存计算架构示意图[71]

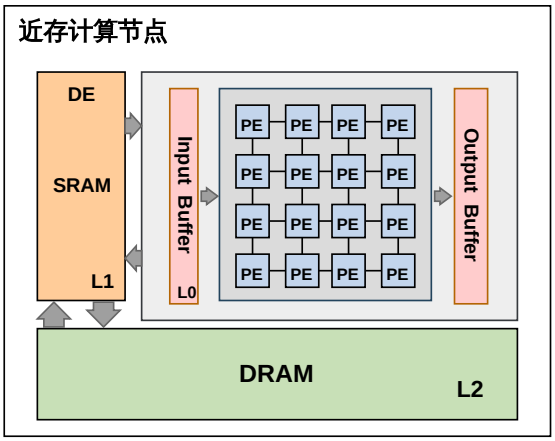


图 2.2　近存计算节点微架构示意图

为片上高速缓存，主要承担数据暂存、重组、访存调度及转发等核心功能。为适配高吞吐量的数据流供给，内部 SRAM 设计支持多端口并发访问机制，允许多个数据块的高效并行读写，从而最大限度地降低缓冲层对计算流水线的阻塞风险。此外，SRAM 的容量配置（如 128 KB、256 KB 或 512 KB）直接决定了片上数据的暂存能力与复用效率。基于数据引擎的缓冲与调度作用，外部存储与计算阵列在访问延迟及吞吐特性上的供需失配问题得以有效缓解，实现了访存与计算之间的高效衔接。

由规则化的处理单元（PE）阵列构成，本文将近存计算节点内的计算单元组织成脉动阵列形式[72]，执行核心的张量运算任务。其计算效能主要受控于阵列规模（如 8 × 8、16 × 16 等）与并行执行能力；而单一 PE 内部的运算资源配置（例如单周期执行 1 次或 2 次乘加运算，MAC）则进一步确立了节点的峰值计算吞吐量。

2.2　Attention 算子负载特征与参数空间

在大语言模型（LLM）的自回归推理过程中，Attention 算子的计算特征表现出显著的阶段性差异，主要分为预填充（Prefill）和解码（Decode）两个阶段[27]。不同阶段的计算强度与访存需求截然不同，这要求 PIM 节点的微架构与数据流划分策略必须具备高度的动态适应能力。

2.2.1　推理阶段的访存与计算特性

预填充阶段（Prefill Phase）：当模型首次接收用户的提示词序列时，输入长度为 $N_{prefill}$。此阶段需要对整个序列进行并行计算，核心操作表现为大规模的矩阵-矩阵乘法（GEMM）。由于计算量随序列长度呈平方级（或接近平方级）增长，该阶段具有较高的算术强度，系统性能通常受限于计算阵列的峰值算力，能够充分利用计算阵列的高吞吐优势[73-74]。

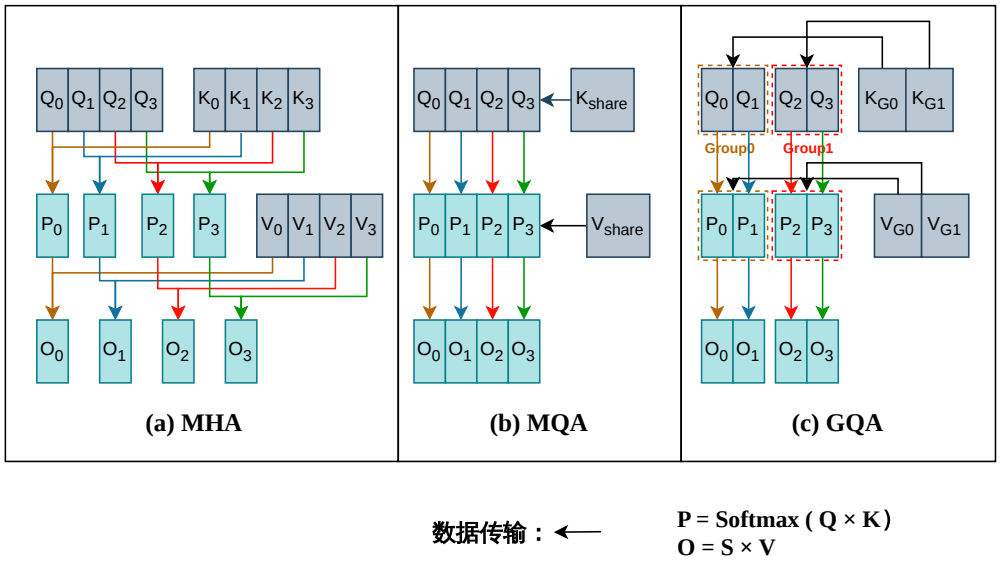
解码阶段（Decode Phase）：模型采用自回归方式逐 Token 生成输出。为了避免对历史上下文的冗余计算，现代 LLM 普遍引入 KV Cache 机制，将历史时刻生成的键（Key）和值（Value）张量常驻于存储系统中[75-76]。在生成第 t 个 Token 时，Query 的序列长度退化为 1，此时注意力分数的计算与聚合过程由 GEMM 退化为矩阵-向量乘法（GEMV）[73,75]。

在 Decode 阶段，每次生成新 Token 仅需极少的 MAC 计算操作，却需要从物理内存中遍历读取极其庞大的 KV Cache 矩阵。这种数据搬运量远大于浮点运算量的非均衡比例，导致计算单元频繁处于等待数据的空转状态，系统表现出极端的访存密集型特征[74,76]。以典型的大模型推理为例，假设批处理大小 $B = 1$，上下文长度 $N = 4096$，若系统需要维持 50 Tokens/s 的流畅生成速度，仅单层 Attention 的 KV Cache 读取就可能消耗数十 GB/s 的内存带宽。传统冯・诺依曼架构有限的片外带宽极易在此成为系统瓶颈，因此采用近存（PIM）架构进行近数据处理的优势就展现了出来。

2.2.2　多种 Attention 变体

为缓解上述 KV Cache 带来的显存容量与内存带宽压力，近年来主流大模型的注意力计算结构发生了重要演进，衍生出不同的变体，进而深刻改变了底层硬件的数据流动与映射需求，如图2.3展示了三种注意力计算的数据流模式。

- **多头注意力（Multi-Head Attention, MHA）**：标准的 Transformer 设计[6]。每个 Query 头都有独立对应的 Key 和 Value 头。其 KV Cache 规模最为庞大，对 DRAM 带宽的施压最为直接，数据在片上 SRAM 中的重用率极低。
- **多查询注意力（Multi-Query Attention, MQA）**：为极简设计，所有 Query 头共享同一组唯一的 Key 和 Value 头[77]。MQA 极大地压缩了 KV Cache



数据传输：←　　P = Softmax (Q × K)
O = S × V

图 2.3　三种注意力计算数据流模式对比

的容量与访存带宽需求，使得在进行注意力计算时，单组 KV 数据可以在片内被多个 Query 头高度重用。然而，这种极其不平衡的 Q 与 K/V 维度映射，对片上数据分发网络（如 NoC 组播）提出了特殊要求。

- **分组查询注意力（Grouped-Query Attention, GQA）**：作为 MHA 和 MQA 的折中方案[78]，GQA 将 Query 头划分为多个组，组内的 Query 头共享一组 Key 和 Value 头。GQA 在维持模型表征能力的同时，实现了较优的推理访存效率。

上述 Attention 变体的存在，意味着 K 和 V 张量的特征维度分布在不同模型中具有高度不确定性。当 PIM 节点处理 GQA 或 MQA 负载时，数据在片上 SRAM（Data Engine）中的驻留价值显著增加；而处理 MHA 负载时，更依赖于优化 DRAM 的条带化突发读取。因此，不存在一套固定的微架构参数与 Tiling 调度能够放之四海而皆准，必须依赖系统化的设计空间探索予以自适应优化。

2.3　多目标设计空间探索理论基础

在确定了近存（PIM）节点的硬件微架构参数与 Attention 算子的负载特征后，设计空间探索（Design Space Exploration, DSE）的核心任务便是在高维、离散与连续混合的参数空间中，寻找最优的架构配置与数据流映射策略。为了科学地指导搜索过程，首先需要明确衡量 PIM 架构优劣的关键性能指标，并建立严谨的多目标优化理论模型。

2.3.1　近存架构的优化目标

在传统的冯·诺依曼架构中，性能瓶颈通常可以用“计算受限（Compute-bound）”或“访存受限（Memory-bound）”来近似界定，并可通过算术强度与可达带宽/算力的关系进行解释[79]。然而，在近存架构下，计算单元（PE/SFU）、片上缓冲（SRAM）与主存（DRAM）高度耦合，其端到端性能表现往往取决于软硬件协同调度与跨层级数据搬运效率[41]。针对大模型推理等数据密集型负载，在进行架构设计空间探索时，学术界与工业界通常会联合关注性能、数据移动与能效等多维指标，并以此刻画可行的权衡空间[54,80]：

- **执行延迟（Execution Latency / Cycles）**：这是衡量系统处理速度最直观的指标。在 PIM 节点内部，总执行周期不仅包含 CGRA 阵列的有效计算周期（由处理单元数量、乘累加延迟与 SIMD 并行度决定），还深切依赖于流水线重叠效率与存储系统可供给的持续带宽。类似地，在面向神经网络推理的专用加速器评估中，端到端时延往往同时受算力与内存系统的共同制约[81]。例如，当片上 SRAM 容量较小时，频繁的 DRAM 数据加载会导致计算单元长时间处于“饥饿”状态，从而使得访存停顿（Stall）周期在总延迟中占据主导地位。
- **DRAM 访存量（DRAM Traffic）**：近存架构的根本初衷是打破“内存墙”，降低片外数据搬运开销。DRAM 访存量反映了计算过程中在 DRAM 与逻辑层数据引擎（Data Engine, DE）之间双向传输的数据总字节数。该指标受微架构分块（Tiling）大小与驻留策略的直接影响。大量 DNN/张量加速器工作已表明，优化数据流与片上重用、降低片外访存，是提升能效与吞吐的关键途径[80,82]。理想情况下，通过优化循环顺序（Loop Order），最大化张量在片上 SRAM 的数据重用率，可以极大地降低 DRAM 访存量。
- **面积与能耗开销（Area and Energy Overhead）**：硬件资源的增加（如更多的 DRAM Bank 通道、更大的 SRAM 容量或更庞大的计算阵列）虽然能提升并行度与数据局部性，但会显著增加芯片的物理面积与静态漏功耗。此外，数据在不同存储层级间的搬移通常是动态能耗的主要来源之一，且 DRAM 访问能耗远高于片上计算与 SRAM 访问[83]。因此，现代加速器/近存架构设计往往将“减少数据移动、提高每比特/每操作的能效”作为核心目标[41,80]。

在实际设计中，上述指标往往存在强烈的互相制约关系（Trade-offs）。例如，增加片上 SRAM 的容量可以有效减少跨层级的 DRAM 访存量，但会直接导致逻辑层面积超标与静态功耗上升；提高计算阵列规模能提升理论峰值算力，但若 DRAM 可持续带宽无法匹配，多余的计算单元将处于空闲状态，反而降低了面

积效率。这类“算力—带宽—能耗”之间的结构性矛盾，在经典性能模型与实际加速器测量中均有体现[79,81]。因此，PIM 架构的设计难以通过单一变量的贪心调节来完成，必须引入多目标优化的数学框架以系统化刻画与搜索可行的权衡解集[54]。

2.3.2　多目标优化问题基本概念

多目标优化问题（Multi-Objective Optimization, MOO）是指在给定的物理或资源约束条件下，同时对两个或两个以上的冲突目标函数进行最小化或最大化的数学问题[84-86]。一般的多目标优化问题可以形式化定义为[85]：

$$\begin{aligned} \min_{\mathbf{x}} \quad & \mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \cdots, f_m(\mathbf{x}))^T \\ \text{s.t.} \quad & g_i(\mathbf{x}) \leqslant 0, \quad i = 1, 2, \cdots, p \\ & h_j(\mathbf{x}) = 0, \quad j = 1, 2, \cdots, q \\ & \mathbf{x} \in \mathcal{X} \end{aligned} \tag{2.1}$$

其中，$\mathbf{x}$ 为设计变量向量（在 PIM 架构探索中对应具体的硬件配置与 Tiling 策略参数），$\mathcal{X}$ 为决策空间（Design Space）；$f_k(\mathbf{x})$ 为第 k 个目标函数（如执行周期、DRAM 访存量等）；m 为目标的数量（通常 $m \geqslant 2$）[86]。

与单目标优化不同，多目标优化的各个目标之间通常存在天然的冲突，导致不存在一个单一的绝对最优解能够同时使所有目标函数达到极值[85-86]。因此，多目标优化理论引入**帕累托支配（Pareto Dominance）**的概念来比较不同解的优劣[84,86]：

对于决策空间中的两个解 $\mathbf{x}_A$ 和 $\mathbf{x}_B$，称解 $\mathbf{x}_A$ 帕累托支配解 $\mathbf{x}_B$（记为 $\mathbf{x}_A \prec \mathbf{x}_B$），当且仅当对于所有的目标维度 $k \in \{1, \cdots, m\}$ 均有 $f_k(\mathbf{x}_A) \leqslant f_k(\mathbf{x}_B)$，且至少存在一个维度 j 使得 $f_j(\mathbf{x}_A) < f_j(\mathbf{x}_B)$[84]。

基于支配关系，若决策空间中不存在任何解能够支配解 $\mathbf{x}^*$，则称 $\mathbf{x}^*$ 为帕累托最优解（Pareto Optimal Solution）。所有帕累托最优解构成的集合称为帕累托最优集（Pareto Set, PS）；将帕累托最优集映射到目标函数空间（Objective Space）中，所形成的曲面或边界即被称为帕累托前沿（Pareto Front, PF）[85-86]，如图2.4所示。设计空间探索的终极目标，便是在可接受的时间开销内，尽可能逼近并均匀覆盖真实的帕累托前沿，以为硬件架构师提供多样化的权衡方案[84]。

2.3.3　多目标优化算法基础

在设计空间探索（Design Space Exploration, DSE）问题中，搜索空间通常具有高维、离散、非凸、不可导以及多局部最优点并存等特点，因此很难直接依赖

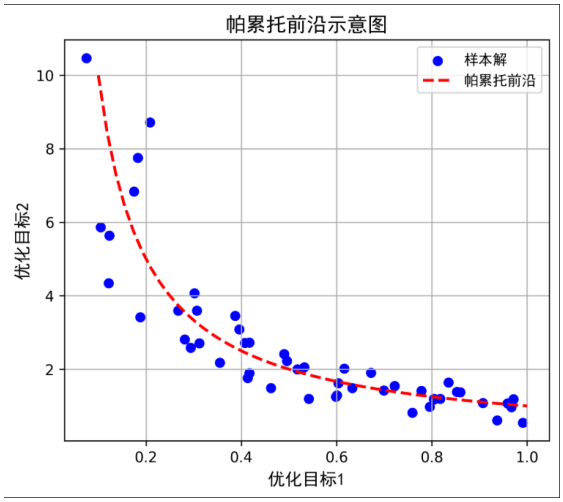


图 2.4　帕累托前沿

传统解析优化方法获得全局最优解。特别是在硬件架构设计中，设计变量既可能包括阵列规模、片上缓存容量、访存通道数等离散参数，也可能包含映射、分块与调度等组合决策，使得目标函数往往呈现出高度复杂的黑盒特征。针对这类问题，学术界提出了多种经典启发式与元启发式优化算法，如模拟退火（Simulated Annealing, SA）、遗传算法（Genetic Algorithm, GA）以及粒子群优化（Particle Swarm Optimization, PSO）等[87-90]。这些方法能够通过随机扰动、群体演化或记忆机制，在复杂搜索空间中有效寻找高质量近似解，因此被广泛应用于组合优化、工程设计优化与硬件 DSE 等场景。

1. 模拟退火方法（Simulated Annealing, SA）

模拟退火是一类基于统计物理退火过程构造的随机优化方法，其基本思想是：将待优化问题的目标函数视为系统能量，通过在高“温度”下允许接受劣化解、在低“温度”下逐步收敛至稳定状态，从而避免搜索过程过早陷入局部最优[87]。设当前解为 $\mathbf{x}$，其目标函数值为 $f(\mathbf{x})$；在其邻域中生成新解 $\mathbf{x}'$ 后，定义目标增量为

$$\Delta f = f(\mathbf{x}') - f(\mathbf{x}).$$

对于最小化问题，若 $\Delta f \leqslant 0$，则新解优于或等于当前解，应直接接受；若 $\Delta f > 0$，则以概率

$$P(\text{accept}) = \exp\left(-\frac{\Delta f}{T}\right)$$

接受该劣化解，其中 T 为当前温度参数[87]。这一接受准则源于 Metropolis 机制，其核心作用在于：当温度较高时，算法更倾向于接受较差解，从而增强全局探索能力；当温度逐渐降低时，算法对劣化解的容忍度减小，搜索行为逐步转向局部开发。实际应用中，温度通常按照某种退火策略递减，例如几何降温形式

$$T_{k+1} = \alpha T_k, \qquad 0 < \alpha < 1.$$

因此，模拟退火算法的关键组成包括初始解生成、邻域定义、接受概率设计以及温度更新策略。对于硬件 DSE 而言，SA 的优点在于实现简单、对变量类型要求低，尤其适合处理离散与组合型设计变量；但其性能对降温速率、初始温度以及邻域扰动方式较为敏感，且在大规模设计空间中收敛速度可能较慢。

2. 遗传算法（Genetic Algorithm, GA）

遗传算法是一类模拟自然选择和遗传机制的群体式随机搜索方法，其基本思想是：将每个候选解编码为一个“染色体”，再通过选择（Selection）、交叉（Crossover）与变异（Mutation）等操作，逐代演化出更优的解集[88,91]。设第 t 代种群为

$$\mathcal{P}^{(t)} = \{\mathbf{x}_1^{(t)}, \mathbf{x}_2^{(t)}, \cdots, \mathbf{x}_N^{(t)}\},$$

其中每个个体 $\mathbf{x}_i^{(t)}$ 对应一个设计点，其适应度函数记为 $F(\mathbf{x}_i^{(t)})$。在比例选择（Fitness-proportionate Selection）中，个体被选中的概率可表示为

$$p_i = \frac{F(\mathbf{x}_i^{(t)})}{\sum_{j=1}^{N} F(\mathbf{x}_j^{(t)})}.$$

被选中的父代个体经过交叉操作产生新后代。例如，在单点交叉中，若交叉位置为 c，则两个父代染色体

$$\mathbf{x}^{(1)} = (x_1, \cdots, x_c, x_{c+1}, \cdots, x_L), \quad \mathbf{x}^{(2)} = (y_1, \cdots, y_c, y_{c+1}, \cdots, y_L)$$

可生成后代

$$\mathbf{z}^{(1)} = (x_1, \cdots, x_c, y_{c+1}, \cdots, y_L), \quad \mathbf{z}^{(2)} = (y_1, \cdots, y_c, x_{c+1}, \cdots, x_L).$$

随后，算法再以概率 p_m 对个体的部分基因执行随机变异，以维持种群多样性并避免早熟收敛。遗传算法的理论基础之一是模式定理（Schema Theorem），其指出：短定义长度、低阶且适应度高于平均水平的模式，在进化过程中会获得指数级增长趋势[88]. 常见形式可写为

$$\mathbb{E}[m(H, t+1)] \geqslant m(H, t)\frac{f(H)}{\bar{f}(t)}\left(1 - p_{\text{disrupt}}\right),$$

其中 $m(H, t)$ 表示第 t 代中模式 H 的个体数量，$f(H)$ 为该模式的平均适应度，$\bar{f}(t)$ 为当代种群平均适应度，p_{disrupt} 为该模式被交叉或变异破坏的概率[88]. 从直观上看，GA 通过“选择优秀解并重组其局部优良结构”的方式实现全局搜索，因此在硬件 DSE 中常用于大规模组合空间探索。其主要优势在于全局搜索能力较强、适合多峰问题；不足之处在于参数较多，如种群规模、交叉概率和变异概率等均会显著影响性能，且在评估成本昂贵时，种群式搜索会带来较高的总计算开销。

3. 粒子群优化（Particle Swarm Optimization, PSO）

粒子群优化是一类受群体协同行为启发的群体智能优化方法。其基本思想是：将

每个候选解看作搜索空间中的一个“粒子”，通过粒子个体经验和群体经验的共同引导，不断更新粒子的位置和速度，从而逼近最优区域[90]。设第 i 个粒子在第 t 次迭代时的位置和速度分别为 $\mathbf{x}_i^{(t)}$ 与 $\mathbf{v}_i^{(t)}$，其历史最优位置为 $\mathbf{p}_i$，全局最优位置为 $\mathbf{g}$，则标准 PSO 的更新公式为

$$\mathbf{v}_i^{(t+1)} = w\mathbf{v}_i^{(t)} + c_1 r_1 \left(\mathbf{p}_i - \mathbf{x}_i^{(t)}\right) + c_2 r_2 \left(\mathbf{g} - \mathbf{x}_i^{(t)}\right),$$

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)},$$

其中，w 为惯性权重，c_1 和 c_2 分别为个体学习因子与群体学习因子，$r_1, r_2 \sim U(0,1)$ 为随机数[90]。从公式可以看出，PSO 中粒子的运动同时受三部分影响：上一时刻的惯性项、朝向个体历史最优位置的认知项，以及朝向群体最优位置的社会项。该机制使得 PSO 在全局探索与局部开发之间形成相对平衡。相比 GA，PSO 不需要交叉与变异操作，算法结构更简洁、参数相对较少，通常在连续优化问题中表现良好；但对于离散变量较多、约束复杂或可行域不规则的问题，需要额外设计离散化映射或修复机制，才能适用于硬件 DSE。

尽管模拟退火、遗传算法和粒子群等启发式算法在全局搜索中表现出色，但它们普遍面临数据效率（Sample Efficiency）低下的缺陷。在近存架构的设计空间探索中，每一个候选解的评估都需要调用周期精确级的底层硬件仿真器，单次评估的时间成本极其高昂。传统启发式算法动辄数以万计的评估次数在实际工程中往往不可接受。为此，基于数据驱动代理模型的贝叶斯优化（Bayesian Optimization, BO）成为解决昂贵黑盒优化问题的主流范式。它通过构建代理模型与获取函数（Acquisition Function），在评估精度与搜索开销之间取得了极佳的平衡。

1. 高斯过程代理模型与期望提升获取函数（EI）

贝叶斯优化的核心在于利用已知的训练数据构建代理模型，并给出预测值的概率分布。在高斯过程（Gaussian Process, GP）模型中，对于给定的数据集 D，对于一个输入 $\mathbf{x}$，其对应的预测值 y 的概率分布遵循如下表达式[92]：

$$p(y \mid D, \boldsymbol{\theta}) = \mathcal{N}(\mu(\mathbf{x}), \sigma^2(\mathbf{x}))$$

其中，$\mu(\mathbf{x})$ 和 $\sigma(\mathbf{x})$ 分别是预测均值和方差的分布，$\boldsymbol{\theta}$ 是高斯模型中的超参数。均值可以视为对未知点的性能预测，而方差可以被视为预测值的置信度（不确定性）。

获取函数的作用正是基于上述概率分布来平衡搜索过程中的“探索（Exploration，倾向于高方差区域）”和“利用（Exploitation，倾向于高均值区域）”。对于一个最小化目标函数 $\min y = f(\mathbf{x})$，传统的单目标优化常采用期望提升（Expected Improvement, EI）获取函数[93]。将 τ 作为当前状态空间中已观测到的最小值，可

以通过计算预测值 y 相对于 τ 的提升量来评估候选点：

$$I(y,\tau)=\begin{cases}\tau-y & y<\tau\\ 0 & \text{否则}\end{cases}$$

只有当预测值 y 小于历史最优解 τ 时才会产生提升。结合高斯模型的概率分布，期望提升的解析表达为：

$$\mathrm{EI}(\mathbf{x})=\mathbb{E}[I(y,\tau)]=\int_{-\infty}^{\infty}I(y,\tau)p(y\mid D,\boldsymbol{\theta})dy$$

算法在每次迭代中通过最大化 $\mathrm{EI}(\mathbf{x})$ 来选出下一个最值得进行真实硬件仿真的候选点。

2. 多目标探索与超体积（Hypervolume）评估指标

上述 EI 获取函数虽然在单目标优化中被广泛应用，但存在无法直接应用于多目标探索过程中的局限性。在多目标场景下，优化的终极目标是逼近由多个相互冲突的目标（如架构的面积开销与执行时间）构成的帕累托前沿。因此，需要引入超体积（Hypervolume, HV）作为评估设计空间中候选点集合效用的核心指标。

对于一个候选解集 $\mathcal{P}$ 和一个参考点 $r=(r_1,r_2)$（一般选取为当前架构设计所能容忍的最差面积和性能边界），超体积定义为计算点集 $\mathcal{P}$ 和参考点 r 在目标坐标系中所围成的体积（对于二维情况即为面积）。其数学表达如下：

$$\mathrm{HV}(\mathcal{P},r)=l_m\left(\bigcup_{p\in\mathcal{P}}[p,r]\right)$$

其中，l_m 为勒贝格测度（Lebesgue measure），$[p,r]$ 代表由前沿点 p 与参考点 r 所围成的超矩形。由于本文的优化目标是最小化架构的面积和执行时间，所围成区域的超体积越大，代表帕累托解集的整体质量越高。

3. 多点期望超体积提升获取函数（qEHVI）

传统获取函数通常只能实现串行评估（一次选取一个候选点），且难以良好地处理多目标与非线性约束。因此，本文在设计空间探索中采用多点期望超体积提升获取函数（Expected q-Hypervolume Improvement, qEHVI）[94]。该方法允许在单次迭代中并行处理并获取 q 个候选点。

基于超体积的定义，当将新采样得到的数据点 y_c 计入到原本的帕累托前沿点集 $\mathcal{P}$ 中时，点集所围成的超体积会产生相应的改变，其产生的超体积提升量（Hypervolume Improvement, HVI）满足以下等式：

$$\mathrm{HVI}(\mathcal{P},r,y_c)=\mathrm{HV}(\mathcal{P}\cup\{y_c\},r)-\mathrm{HV}(\mathcal{P},r)$$

qEHVI 算法通过在高斯代理模型的后验联合预测分布上计算上述提升量的数学期望，从而在复杂混合变量空间中联合评估一批候选参数组的潜在贡献。最大化 qEHVI 能够在严格控制底层仿真器调用次数的前提下，引导搜索过程快速拓展并均匀覆盖 PIM 架构的帕累托前沿。

2.4　本章小结

本章围绕近存节点的设计空间探索基础展开，首先介绍了基于 DRAM 的 PIM-Node 架构及其存储子系统、数据引擎和计算子系统的关键参数化设计，分析了这些参数共同构成的复杂硬件设计空间；随后结合 Transformer 推理中 Prefill 与 Decode 阶段的计算与访存特性，以及 MHA、MQA、GQA 等 Attention 变体对数据流与 KV Cache 访问模式的影响，说明了不同负载对 PIM 架构配置的差异化需求；在此基础上，引入多目标设计空间探索的理论框架，阐述执行延迟、DRAM 访存量及面积能耗等优化目标之间的权衡关系，并概述模拟退火、遗传算法、粒子群优化及基于高斯过程的贝叶斯优化方法，为后续开展面向 Transformer 推理的 PIM 架构设计空间探索提供了理论基础。

第 3 章　近存架构 Attention 推理的层次化数据流映射方法

3.1　注意力推理负载表示方法

本节从执行阶段、头部组织和规模特征三个方面，对 attention 数据流的关键负载特征进行梳理，并

3.1.1　注意力计算的负载特征表示

自注意力机制（Self-Attention）是 Transformer 架构中的核心计算单元，其基本计算形式可表示为

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \tag{3.1}$$

其中，Q、K 和 V 分别代表查询（Query）、键（Key）与值（Value）张量，d 为特征维度。从执行层面来看，注意力计算通常涵盖相关性分数计算、归一化处理以及上下文聚合三个核心子算子。尽管这一理论公式在不同模型中是统一的，但在实际推理场景中，其底层执行行为会随着运行阶段与架构拓扑的不同而产生巨大差异。对于近存计算（PIM）架构的设计与映射分析而言，核心关注点不仅在于公式本身，更在于不同负载形态所衍生的计算密度、数据吞吐需求以及片上中间结果生命周期的动态变化。

首先，注意力推理负载表现出极强的阶段差异性。大语言模型推理全过程通常被划分为预填充（Prefill）与解码（Decode）两个执行阶段。预填充阶段面向完整的输入序列，查询、键与值张量整体参与运算，具备较高的数据与算子并行度；其计算模式高度近似于规则的稠密矩阵乘法，因而高度依赖多维分块与批量并行策略，呈现出典型的计算密集型特征。相反，解码阶段面向自回归生成过程中的单步或小批量词元（Token），此时系统仅输入当前步的查询向量，而键与值数据主要依赖对历史缓存的读取。随着生成序列的延长，注意力计算逐渐退化为以单条查询向量为中心的矩阵-向量乘法，算术强度相比于访存需求急剧下降，而读取历史键值数据所引发的访存开销则持续增大。因此，解码阶段在实际硬件执行中表现出显著的访存受限（Memory-bound）特性。

其次，不同的头部组织架构直接决定了底层数据的复用与映射特征。标准的多头注意力（MHA）采用查询头与键值头一一对应的严格绑定机制，每个注意力头均需独立读取与维护键值表示，这不可避免地带来了庞大的键值缓存（KV Cache）开销。为缓解解码阶段的片外带宽压力与片上缓存瓶颈，业界相继引入了多查询注意力（MQA）与分组查询注意力（GQA）等变体。MQA 允许多个查

询头共享全局唯一的一组键值头，从而大幅削减了外部读取量；GQA 则采用分组共享机制，在模型表达能力与带宽约束之间取得了较好的折中。这类结构变异对于硬件映射的核心影响在于，它重塑了查询头与键值头之间的空间拓扑，进而深刻改变了数据在片上网络中的广播机制、复用率以及处理单元（PE）集群的组织形态。

多尺度规模属性构成了约束系统性能的关键物理边界。影响硬件调度的核心参数涵盖了批次大小（Batch Size）、查询序列长度、键值序列长度、头部数量及其特征维度等。与常规计算不同，注意力推理的数据流转高度受控于持续膨胀的键值缓存。在解码阶段，单步查询触发的计算量极为有限，但其关联的键值读取量却随上下文长度呈线性增长，这使得系统的峰值性能极易触及数据供给路径与存储带宽的上限。进一步地，不同头部拓扑对访存带宽的挤压程度各异：如图 2.3 所示，(a) MHA 模式下各头需独立加载外部数据，带宽压力极高；(b) GQA 通过分组机制实现了局部数据的片上重用；(c) MQA 则支持全局数据的跨核广播，最大程度抑制了片外访存流量。

除上述阶段特征、结构特征和规模特征外，注意力机制内部存在着严密的算子间时空依赖。相关性分数计算、归一化处理与上下文聚合首尾相接，软硬件层面的不同调度策略会影响中间结果的生命周期边界。这些中间结果张量既可以在片上高速缓存中短暂驻留并被后续算子即时消费，也可能因中间存储容量受限而被写回至 DRAM 主存，进而产生二次 DRAM 访存开销。

综上所述，面向 PIM 近存计算架构的注意力推理负载，单一维度的理论计算量或单纯的层级访存量统计，已无法真实反映系统的执行瓶颈。因此，必须构建一种能够全面涵盖上述阶段、结构化复用差异及时序依赖的统一参数化模型。下一节将基于上述多维约束，正式建立面向硬件映射分析的统一负载参数表示方法，为后续的数据流设计与空间探索奠定分析基石。

3.1.2　Attention 负载编码框架

为统一表示不同执行阶段、不同头部组织方式以及不同缓存状态下的注意力推理计算负载，本文构造注意力负载编码框架。整个过程可以归纳为以下几个步骤：

（1）确定负载的执行阶段。对于大语言模型推理，Attention 计算通常处于 Prefill 或 Decode 两种阶段之一。执行阶段决定了当前负载的输入形式、上下文范围以及数据访问模式。记阶段变量为

$$M \in \{\text{Prefill}, \text{Decode}\} \tag{3.2}$$

其中，Prefill 表示完整上下文上的整体计算，Decode 表示自回归生成过程中的增量计算。

（2）确定负载的头部组织方式。不同 Attention 结构的差异主要体现在 Query head 与 Key/Value head 的对应关系上。记结构变量为

$$T \in \{\mathrm{MHA}, \mathrm{GQA}, \mathrm{MQA}\} \tag{3.3}$$

并记 Query head 数量与 KV head 数量分别为 H_q 与 H_{kv}。则不同结构可统一写为由 (H_q, H_{kv}) 所刻画的头部组织形式。该表示给出了负载内部的数据共享关系与并行组织关系。

（3）提取负载的规模参数与缓存状态。在确定阶段属性与结构属性之后，进一步提取描述单个负载实例的核心参数，包括批次规模、序列范围、头部维度以及缓存状态。由此，单个 Attention 负载实例可表示为参数集合

$$\mathcal{W} = \{M, T, B, Q_{len}, KV_{len}, D_h, H_q, H_{kv}, L_{cache}, L_{append}, W_{dtype}\} \tag{3.4}$$

其中，B 表示批次大小，Q_{len} 表示当前 Query 长度，KV_{len} 表示参与计算的 Key/Value 长度，D_h 表示单头维度，L_{cache} 表示历史 KV Cache 长度，L_{append} 表示当前步追加长度，W_{dtype} 表示数据类型宽度。

在 Decode 场景下，当前可见上下文长度由历史缓存与当前追加部分共同决定，因此有

$$KV_{len} = L_{cache} + L_{append} \tag{3.5}$$

该式给出了 Decode 过程中负载状态随上下文增长而变化的基本形式。

（4）生成负载的结构化计算图。在参数化描述的基础上，进一步构造对应的 WorkloadDAG。记其为

$$G = (V, E) \tag{3.6}$$

其中，V 表示节点集合，E 表示边集合。节点集合由输入张量、中间张量、输出张量以及核心计算操作共同组成，即

$$V = V_{tensor} \cup V_{op} \tag{3.7}$$

边集合用于表示张量在不同算子之间的生产与消费关系，即

$$E \subseteq V \times V \tag{3.8}$$

对于给定负载 $\mathcal{W}$，其结构化表示可写为

$$G_{\mathcal{W}} = \Phi(\mathcal{W}) \tag{3.9}$$

其中，$\Phi(\cdot)$ 表示从参数化负载描述到结构化计算图的映射过程。通过该映射，参数层中的阶段信息、结构信息、规模信息与缓存状态被进一步展开为具有明确执行语义的数据流表示。

（5）形成统一的负载表示。综合上述步骤，单个 Attention 负载可表示为参数集合与结构化计算图组成的二元组

$$\mathcal{A} = (\mathcal{W}, G_{\mathcal{W}}) \tag{3.10}$$

其中，$\mathcal{W}$ 用于描述负载的抽象属性，$G_{\mathcal{W}}$ 用于描述负载的执行结构。由此，Attention 负载被统一表示为同时包含阶段信息、结构信息、规模信息与依赖信息的分析对象。

基于上述编码方式，Prefill 与 Decode 两类执行阶段、MHA、GQA 与 MQA 等不同头部组织方式，以及 KV Cache 持续增长条件下的负载变化，均可纳入同一表示框架之中，从而为后续的任务划分、数据放置以及存储级绑定的层次化映射分析提供统一输入。

3.2　近存计算架构下注意力计算数据流映射方法

本节提出一种面向 PIM 节点的 Attention 数据流与执行结构表示方法。该方法能够同时表达任务分块、数据驻留和执行顺序，由 placement 方案、存储级数据分块（tiling）和层次化执行树的生成三部分组成：其中，placement 方案给出张量在存储层级中的驻留与边界语义约束；存储级 tiling 确定当前存储级绑定的张量的循环分块、数据复用与写回边界；在此基础上得到的映射候选构建成细粒度执行树，以供后续分析与评估过程统一使用,Attention 数据流计算示意图如图3.1。

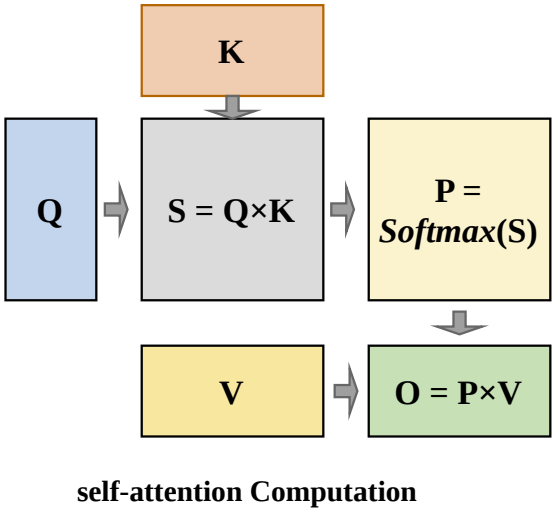


图 3.1　注意力计算数据流

3.2.1　注意力算子分组融合方法

在近存计算架构下，算子间数据传输通常需要经过受限的片外存储层。对于解码侧注意力，其计算过程可表示为一张算子级有向无环图，如图 3.2 所示，

输入张量一方面直接参与 QK MatMul，另一方面分别经过 K Append(Key 追加)、K Trans(Key 转置) 和 V Append(Value 追加) 后进入后续计算。其中，主要计算 $Q \times K \rightarrow \text{Softmax} \rightarrow P \times V$ 具有严格的数据依赖关系：QK MatMul 产生的中间结果 Scores(注意力分数) 作为 Softmax 的输入，Softmax 输出的 Probs(概率) 进一步作为 PV MatMul 的输入。若采用单算子独立映射方式，则上述中间激活张量需要在算子边界之间频繁搬移，易形成显著的片外带宽压力。

为降低跨算子数据搬移开销，本文在数据流映射之前引入算子分组融合策略。依据算子间的依赖关系与语义属性，使原本跨算子显式传递的中间结果尽可能转化为组内临时张量，并优先驻留于片上缓冲之中，从而降低访存开销。融合后，映射阶段的基本对象不再是单算子，而是具有明确组输入、组输出及组内临时张量的数据流子图。图 3.2 给出了一个典型的分组结果：其中，K Append 与 K Trans 为数据预处理组，V Append 单独为一组，而 QK MatMul、Softmax 与 PV MatMul 构成融合计算组。

1. 算子与张量的语义分类

在算子层面，本文依据其在注意力执行过程中的职责，将算子划分为前端阶段与核心阶段两类。前端阶段主要包括 KVAppend、CacheView(缓存视图)、Transpose 和 HeadBroadcast(头维广播)，核心阶段包括 QK、Softmax 和 PV。该划分用于约束融合组内部的算子组合，避免不同执行职责的算子在同一融合组内无约束混合。

在张量层面，本文根据其在数据流中的角色，将张量划分为状态张量、输入张量、激活张量、输出张量和统计张量。该划分用于区分组内可闭合保留的数据与必须显式暴露的边界数据，从而为融合边界构造和映射接口生成提供依据。

2. 融合策略合法性约束

设原始注意力计算图为 $G = (V, E)$，对任意候选融合子图 $S \subseteq V$，需满足以下约束：

（1）片上缓存容量上界约束

$$|S| \leqslant N_{\max},$$

其中，$N_{\max}$ 表示允许的最大组规模。该约束用于保证融合组规模不超过片上资源承载能力。

（2）DAG 凸性约束若 $u, v \in S$，则从 u 到 v 的依赖路径不能经过组外节点，即

$$\forall u, v \in S, \quad \text{Path}(u, v) \subseteq S.$$

该约束保证融合组内部依赖闭合，并维持原图拓扑关系不变。

（3）执行阶段一致性约束同一融合组中的算子需具有一致的阶段属性，前端

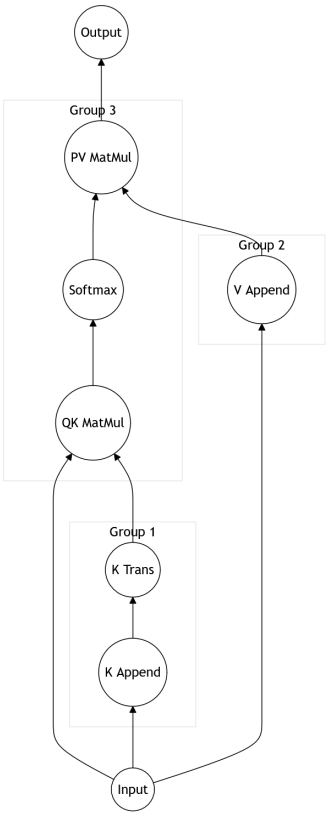


图 3.2　解码侧注意力算子分组融合计算示意图

阶段算子与核心阶段算子不可直接混合。

（4）边界张量语义约束任一融合组的输出边界上不允许同时出现状态张量和最终输出张量，以保持组外接口职责单一。

此外，所有融合组需构成原始算子集合上的互不重叠划分，且融合后形成的组级依赖图仍保持有向无环结构，以保证算子唯一归属和组级执行顺序的可拓扑化。

3. 融合组的映射方法

经过上述约束筛选后，原始注意力计算图被重构为由算子融合子图组成的计算图，保留组与组之间必要的边界张量，而将组内可闭合的数据依赖转化为局部临时张量，从而为后续数据流映射提供统一接口。

在组内，原先跨算子传播的激活张量可转化为组内临时张量，并优先分配至片上缓存单元中。以注意力计算主链为例，当 QK、Softmax 和 PV 分属不同执行单元时，SCORES 与 PROBS 表现为显式边界张量；当三者被纳入同一融合组后，这两类中间激活即可转化为组内临时数据，仅在局部范围内完成生产、消费与释放，从而减少片外写回与重复读取。融合结果还将直接影响组间执行关系的构造。若相邻融合组之间仅传递维度匹配的纯激活张量，则可进一步组织为流水绑定；若涉及状态张量、输出张量或统计张量，则采用顺序执行方式。

需要说明的是，本文提出的算子分组融合方法并不预设唯一最优的分组结

果，而是在语义约束与结构约束共同作用下，为原始注意力计算图生成一组合法的融合候选。图 3.2 仅展示其中一种典型分组形式，用于说明该方法如何将原始算子级 DAG 转换为适合映射的组级 DAG。不同候选方案在边界张量规模、组内临时张量比例以及执行绑定方式等方面存在差异，其优劣仍需结合后续映射与仿真结果进行综合评估。因此，该方法的作用在于缩减设计空间并提供高质量融合基元，而非在本阶段直接确定最终最优方案。

3.2.2　注意力算子 placement 模式表示方法

传统的数据 placement 通常将张量 tile 在不同存储层级中的驻留关系视为静态属性，即仅描述“某个 tile 位于哪一级存储中”。对于常规加速器上的映射优化，这种表示通常已经足够；但对于 PIM-attention，尤其是 decode 场景，仅刻画静态驻留位置并不能完整反映数据组织方式对性能的影响。其原因在于，系统性能不仅取决于张量“放在哪里”，还取决于张量在不同执行阶段是否需要持续保留、在层次边界处触发何种数据动作，以及这些动作如何与 KV cache 等长生命周期对象相协调。

为此，本文将传统 placement 扩展为一种语义化的数据 placement 表示。该表示不再仅描述张量的静态驻留位置，而是进一步显式建模粗粒度 attention 数据流中不同类型数据对象在各存储层级中的驻留关系与边界行为。这里的数据对象包括输入操作数（如 Q, K, V）、中间结果（如 Scores、Stats、Probs、Partial-O）、最终输出 O，以及 decode 阶段具有跨步长生命周期的 KV Cache。由于 PIM 节点的执行效率高度依赖 memory-side 的数据保留、重载、写回与驱逐策略，因此这种带有生命周期语义的 placement 表示比传统静态 placement 更适合刻画 PIM-attention 的映射空间。

具体而言，本文采用两类基本语义来定义一个 placement 模式。

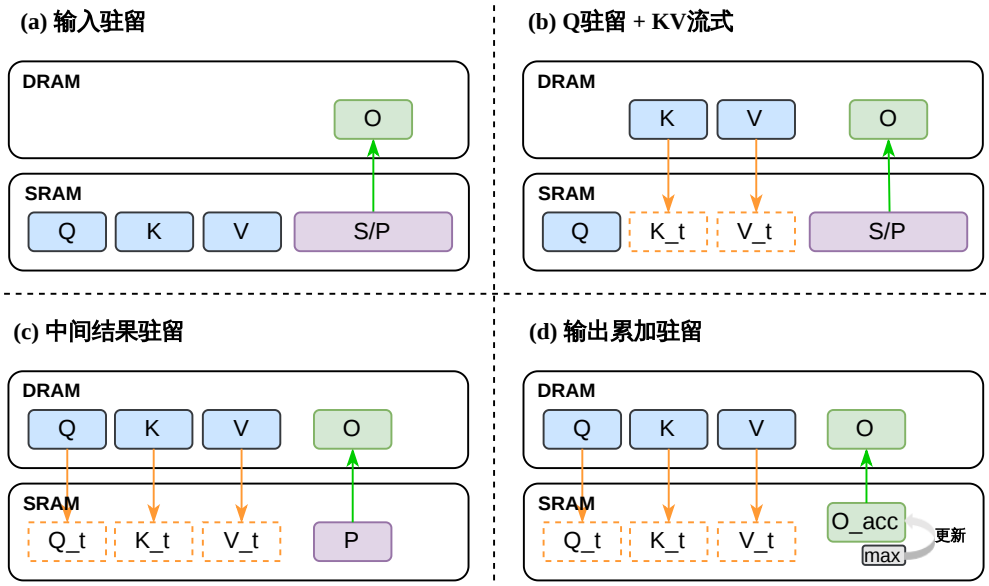
（1）驻留集合（Residency Set）驻留集合用于描述哪些类型的数据对象在给定存储层级上被保留。与传统仅按张量名称区分的方式不同，本文按照粗粒度 attention 数据流中的角色对数据进行分类，包括 Query、Key、Value、Scores、Stats、Probs、Partial-O、Output 和 KV Cache 等。对于每一类数据对象，placement 模式给出其在 DRAM、SRAM 以及更细粒度局部范围中的驻留位置。驻留集合刻画的是数据对象在 memory hierarchy 中的基本组织方式，即“哪些数据被留在近端，哪些数据以流式方式经过”。

（2）边界动作（Boundary Actions）边界动作用于描述数据对象在局部作用域边界处发生的数据迁移与生命周期转换行为。例如，当某类数据进入某一级局部存储时，可能需要执行加载或预取；当其离开当前作用域时，则可能触发写回、保留或驱逐。边界动作使 placement 表示不仅包含“存放位置”，还包含“在

阶段切换或作用域边界上如何处理”的语义。对于 decode 场景下的 KV Cache，这一点尤为关键，因为其性能影响主要来自跨步长的保留、复用与搬运策略，而不是某一次计算中的瞬时驻留位置。

基于上述两类语义，本文将一个 placement 模式表示为：**若干数据角色在不同存储站点上的驻留关系，以及这些角色在站点边界处对应的数据动作集合**。这种表示方式并不直接等价于最终的执行结构，而是作为后续映射搜索的先验约束，用于限定某一类合法的数据组织方式。换言之，placement 模式首先定义粗粒度 attention 数据流在存储层次中的基本布局；在此基础上，后续的存储级 tiling、算子融合与树状映射构造只能在该布局允许的子空间内展开。因此，placement 不再是映射结果的附属属性，而是划分映射搜索空间的首要维度之一。

在此基础上，本文构造若干典型 placement 模式，以覆盖不同的数据复用倾向与数据流组织策略，如图3.3所示。这些模式并非对某个具体执行实例的逐条枚举，而是对若干代表性 placement 子空间的抽象概括。例如，一类模式强调输入操作数（如 Q, K, V）在片上存储中的保留，以降低边界搬运；另一类模式强调输出或部分和在局部范围内持续累积，以减少中间结果的反复写回；还有一类模式面向 decode 阶段，将 KV Cache 及其相关中间对象纳入统一的保留/流式处理策略，以适配长生命周期状态对象的复用特征。通过这些模式，搜索过程可以避免在全空间上进行无结构枚举，而是首先在若干具有明确语义差异的子空间之间进行筛选。



$$Q \times K^T \rightarrow S \rightarrow P = Softmax(S) \rightarrow P \times V \rightarrow O$$

图 3.3　典型数据 placement 模式

形式上，一个 placement 模式可表示为

$$P = (\mathcal{R}, \mathcal{T}),$$

其中，$\mathcal{R}$ 为驻留关系集合，$\mathcal{R} \subseteq Role \times Site$，用于描述不同数据角色在不同存储站点上的驻留关系；$\mathcal{T}$ 为边界动作集合，$\mathcal{T} \subseteq Role \times Site \times Boundary \times Action$，用于描述相应数据角色在站点边界处发生的加载、预取、写回和驱逐等动作。该表示使 placement 从单纯的静态驻留描述扩展为带有生命周期语义的数据组织表示。

后续映射构造过程并不直接在原始操作空间上枚举所有 placement 组合，而是先基于 placement 模式生成合法的数据组织候选，再结合 tiling 与执行树结构进一步实例化为具体映射。

3.2.3　存储层级绑定的 Tiling 表示方法

在 Placement 方案已经确定张量驻留层级和中间结果数据驻留边界之后，仍需进一步解决在既定数据放置约束下，Attention 任务应如何被划分为适合 PIM 节点执行的细粒度子任务。对于面向 PIM 节点的映射问题而言，任务划分并不能仅通过单一 tile 尺寸完成，而需要结合不同存储层级逐步确定局部计算范围、循环组织方式以及局部复用范围。因此，本文采用基于存储层级的 Tiling 表示方法，将细粒度任务划分描述为不同存储层级上的局部 tile 节点。

（1）存储层级上的 tile 的分块范围。本文首先将 Attention 任务的细粒度划分建立在存储层级之上，而不是直接建立在单层计算块之上。当前实现中，任务划分沿 DRAM、SRAM 和 PE 三个层级依次展开。对每一层级，都定义一个局部 tile，用于描述该层级在 Query 侧和 Key/Value 侧所覆盖的局部计算范围。这样，原始逻辑循环空间不再以全局方式一次性展开，而是被逐层收缩为与不同存储层级相匹配的局部任务范围。外层 DRAM tile 决定一次外存搬运所覆盖的局部区域，SRAM tile 进一步在片上缓冲约束下细化计算范围，PE tile 则给出最细粒度的执行块。

（2）存储级上 tile 的循环顺序。在给定局部 tile 范围之后，仅知道分块大小仍不足以确定细粒度任务的执行方式。为此，本文进一步在每一层 tile 表示中显式给出循环组织顺序，用于描述当前层级内各维度的遍历次序。当前实现中，设计空间主要围绕 Query 侧和 Key/Value 侧两个核心维度展开，并在内部统一映射为局部循环维度。由此，tile 表示不再只是局部尺寸的集合，而成为“分块范围 + 循环顺序”的联合描述。这种联合表示使任务划分结果不仅能够反映局部工作集大小，还能够进一步反映局部执行次序及其对数据复用方式的影响。

（3）tile 的时空复用。除分块范围与循环顺序外，本文还进一步区分各层级 tile 的执行属性，即当前层级是以时间复用方式展开，还是进一步以空间并行方式映射到下层计算资源。对于外层存储层级，该属性主要用于区分局部任务在时间上的串行展开方式；对于片上层级和计算层级，则进一步决定局部任务是否以空间并行方式落到计算阵列。

（4）将多层 tile 统一为组级 Tiling 表示。不同存储层级上的 tile 并不是彼此孤立的。为了避免分层分块信息分散，以 group 为单位组织多层 tile，并将其统一表示为组级 Tiling。同时包含各层级上的局部 tile 描述以及组内操作组织方式。组间执行方式主要包括串行（sequential）和流水线（pipline）两类。前者表示组内操作依次执行，后者表示相邻局部任务在一定范围内形成重叠执行。

经过上述步骤后，Attention 任务在不同存储层级上被逐步展开为一组具有明确局部范围、循环组织顺序和执行属性的细粒度子任务。这些子任务并不直接作为设计空间探索的输入，而是作为后续层次化映射结构生成的输入。通过将不同存储层级上的局部范围、循环组织顺序、空间/时间映射属性以及组内执行方式统一纳入同一表示框架。Attention 任务在既定 Placement 方案约束下被逐步展开为适合 PIM 节点执行的细粒度子任务，作为后续层次化映射结构的基础。

3.2.4　层次化映射树的生成方法

将通过 fusion，placement，tilling 得到的映射候选进一步实例化为一种层次化映射树，用于统一表示 Attention 任务在近存计算节点上的数据驻留方式、局部分块结构、循环展开关系及边界语义。映射树中的边表示层次包含关系，而不表示算子之间的数据依赖关系；原始依赖关系仍由工作负载 DAG 及融合后的组间拓扑关系维护。

对于给定负载，确定当前的 Fusion、Placement 与存储级 Tiling，并结合硬件上下文形成完整的映射候选，再将其展开为一棵具有明确层次边界的映射树。树根对应整个负载在当前近存计算节点上的全局执行范围，其下逐层组织 group 级作用域、存储层级上的局部 tile、tile 内部的循环结构以及最内层具体执行的算子实例。

映射树中的节点用于分别表示控制范围、局部分块、循环顺序、张量驻留、边界动作和具体操作等语义。将节点分为以下几类：

- 作用域节点（scope node）用于表示算子融合 group 级的控制范围，是组织局部结构的外层边界；
- tile 节点（tile node）用于表示某一存储层级上的局部计算范围，记录该层级的分块尺寸、循环顺序及时间/空间映射属性；循环节点用于显式表示 tile 内部的局部遍历顺序及嵌套关系；
- 存储节点（storage node）用于表示张量在某一层级上的驻留状态及局部生命周期；
- 动作节点（action node）用于表示读、写、驱逐等边界相关行为；
- 算子节点（op node) 则表示当前局部范围内实际执行的最小计算操作。

映射树的生成首先根据 Fusion 结果确定 group 的划分方式及组间顺序，在

树中建立程序级根节点和各 group 对应的作用域节点；随后，结合 Placement 方案，为每个 group 解析其输入张量、中间结果和输出张量在不同存储层级上的驻留关系、生命周期边界及边界动作，并将这些语义附着到相应作用域或 tile 节点上；在此基础上，再根据存储级 Tiling 参数，依次展开 DRAM、SRAM 等层级上的局部 tile，并进一步按照给定循环顺序生成 tile 内部的循环嵌套结构；最后，在最内层局部范围内挂接具体算子节点，从而得到完整的层次化映射树。

3.3　本章小结

本章围绕近存计算节点上的 Attention 推理映射问题，提出了面向 PIM 架构的层次化数据流表示方法。首先，从执行阶段、头部组织方式、规模参数以及阶段间依赖等角度，对 Attention 推理负载的关键特征进行了归纳，建立了能够统一描述 Prefill/Decode、MHA/GQA/MQA 及 KV Cache 状态的参数化负载编码框架，并进一步将其展开为带有张量语义和阶段依赖关系的 WorkloadDAG。随后，面向近存计算节点，构建了由 Placement 模式、存储层级绑定的 Tiling 表示以及层次化映射树组成的数据流表示体系。其中，Placement 用于刻画张量在不同存储层级中的驻留关系与边界动作，存储级 Tiling 用于描述不同层级上的局部分块范围、循环顺序与时空映射属性，层次化映射树则对数据驻留、局部分块、循环展开和算子执行进行统一组织。通过上述方法，本文为后续面向 PIM 节点的 Attention 映射空间构造、代价分析与优化搜索提供了统一、结构化的表示基础。

第 4 章　近存计算架构软硬件协同设计空间探索

4.1　近存计算架构设计特征表示

4.1.1　关键参数

基于本文 2.1 节对于近存计算节点架构的分析，逻辑层面的一个子阵列与底层的若干 DRAM Bank 组成一个完整的功能单元，定义为一个近存计算节点。一个近存计算节点由本地 DRAM 存储器，包含缓存的存储控制器以及计算阵列等模块组成。其中本地存储 DRAM bank 的结构决定了节点内部有效带宽和容量的上限；片上缓存容量影响着数据流计算中的数据重用策略，而偏上计算阵列的规模和算力大小决定了存算节点的峰值算力。上述三个模块牵涉到的若干参数存在负载的非线性耦合：例如，增加 DRAM bank 数量可能提升带宽，但如果缓存容量不足，则数据重用效率无法充分发挥；同样，增加计算阵列规模提升算力潜力，但在带宽受限或缓存不匹配的情况下，其性能提升可能远低于理论峰值。因此，在设计近存计算节点时，必须综合考虑存储、缓存和计算模块的协同效应，优化参数配置以最大化节点整体性能。

1. 存算节点的存储配置

近存计算节点的存储结构是系统设计的核心，其容量与组织方式直接影响节点的带宽和计算性能。基于二维阵列布局特点，本文提出单节点存储容量的计算方法如下：

设系统总 DRAM 容量为 C_{tot}，每个 DRAM Bank 的存储容量为 C_{bank}，节点阵列横向尺寸为 W_{arr}、纵向尺寸为 H_{arr}，每个节点内部包含的 DRAM Bank 数量为 B_{node}，则系统总存储容量可表示为：

$$C_{\text{tot}} = B_{\text{node}} \cdot C_{\text{bank}} \cdot W_{\text{arr}} \cdot H_{\text{arr}} \tag{4.1}$$

在物理实现中，由于阵列规模可以扩展，总容量 C_{tot} 与芯片面积呈线性关系。同时，节点 Bank 数量与阵列尺寸之间存在反向约束关系：

$$B_{\text{node}} = \frac{C_{\text{tot}}}{C_{\text{bank}} \cdot W_{\text{arr}} \cdot H_{\text{arr}}} \tag{4.2}$$

该关系表明：扩展阵列尺寸有助于提升整体并行度，但会导致每个节点内 Bank 数量减少，从而降低单节点的存储带宽；相反，增加单节点 Bank 数量可以构建高密度节点以减小跨节点通信开销，但若高位宽 DRAM 接口与访问模式不匹配，带宽利用率可能受限，性能无法达到理论上限。

在确定单节点 Bank 数量 B_{node} 后，其组织方式对有效位宽有显著影响。** 同步并行模式 **（LockStep）通过并联位线实现 Bank 位宽叠加，其总位宽为：

$$BW_{\text{tot}}^{(S)} = B_{\text{node}} \cdot BW_{\text{bank}} \tag{4.3}$$

其中 BW_{bank} 表示单 Bank 位宽。而 ** 独立控制模式 ** 保持各 Bank 独立位宽，并通过多路复用器（MUX）选择访问 Bank，其有效总位宽为：

$$BW_{\text{tot}}^{(I)} = BW_{\text{bank}} \tag{4.4}$$

设计时应结合应用的内存访问特性：对于高密度、连续的数据访问，推荐同步并行模式以充分利用高带宽；对于稀疏或随机访问，独立控制模式更能提升带宽利用率并降低访问冲突。

2. 存算节点的缓存配置 SRAM 缓存作为计算单元阵列与 DRAM 之间的核心数据缓冲层，对数据复用和能效优化起着关键作用。尽管近存计算架构通过近 DRAM 存储显著提高了带宽性能，但在执行数据密集型任务（如 Transformer 或其他神经网络计算）时，DRAM 带宽仍可能无法完全满足计算单元的吞吐需求。SRAM 缓存的能效优势主要体现在两个方面：单位访问能耗远低于 DRAM，一般降低 1–2 个数量级；通过缓存中的数据复用策略，可显著减少 DRAM 的访问次数。在深度神经网络计算中，由于多层循环嵌套的特性，输入特征张量、权重参数及输出结果在时间和空间维度上均具有复用机会。缓存容量的大小直接决定了数据复用的深度和效率，因此缓存配置成为硬件优化设计的重要组成部分。同时，缓存系统设计受到物理约束的制约：一方面，近存计算节点的芯片面积限制了 SRAM 的总容量；另一方面，增加缓存容量会带来静态漏电功耗和动态访问能耗的上升，当容量超过某一阈值时，缓存自身的能耗可能抵消其带宽优势，从而降低系统整体能效。由于多头注意力计算中输入、权重和输出的访问模式具有特定的时间和空间复用特性，缓存容量应针对 Attention 负载进行优化配置，以实现存储资源与计算单元吞吐的高效匹配，并提升系统能效。

3. 存算节点的计算阵列配置近存计算节点的峰值计算能力与其内部计算单元矩阵的规模直接相关。增加计算单元数量能够显著提高节点的理论算力，但受限于芯片面积与工艺约束，计算单元的集成密度存在上限。同时，计算单元矩阵的布局结构会影响数据流路径和硬件利用率。以 Transformer 模型的多头注意力计算为例，不同矩阵拓扑会导致计算资源利用效率存在差异。例如，当阵列采用 $Row_{\text{PE}} \times Col_{\text{PE}} = 4 \times 4$ 时，如果注意力头数不是 4 的整数倍，矩阵边界单元可能出现周期性空闲，导致部分硬件资源未被利用；而当阵列为 2×8 布局时，在头数不足 8 的整数倍情况下，也会出现类似的闲置问题。为了便于量化分析，本文

采用 $(N_{\text{row}}, N_{\text{col}})$ 表示计算单元阵列的尺寸，其中 N_{row} 为行方向的并行单元数，N_{col} 为列方向的并行单元数。通过调整 $(N_{\text{row}}, N_{\text{col}})$，可以评估不同矩阵拓扑对峰值算力、单元利用率及数据传输效率的影响，为存算节点的计算单元设计与优化提供指导。

4.1.2　近存计算节点的软硬件表征模型

1. 硬件表征模型：近存计算架构的硬件设计需重点考量存算节点存储配置、计算单元阵列规模及片上缓存容量这三类核心参数的协同优化。本文搭建的近存计算硬件架构的详细设计参数列于表4.1所示。

表 4.1　单节点近存计算架构硬件设计参数

设计参数	描述
Row_{PE}	节点内计算单元阵列的行数
Col_{PE}	节点内计算单元阵列的列数
MAC_{PE}	单个 PE 每周期可执行的 MAC 数量
C_{SRAM}	节点内 SRAM 缓存容量
BW_{SRAM}	节点内 SRAM 缓存总带宽
C_{DRAM}	节点内 DRAM 的存储容量
BW_{DRAM}	节点内 DRAM 的总带宽
Map_{DRAM}	节点内 DRAM 的地址映射方式

4.2　近存计算节点软硬件协同设计空间探索

4.2.1　整体流程

4.2.2　设计空间建模

1. 设计空间规模与复杂度分析

为系统刻画 PIM 节点内 Attention 映射的软硬件协同优化问题，本文将联合设计空间划分为硬件参数空间、融合参数空间、placement 参数空间以及 tiling 参数空间四个部分。四类参数分别对应底层资源供给能力、算子协同执行方式、数据与计算的放置策略，以及多层循环分块与映射方式。由于各子空间内部均包含多个离散候选值，因此整体搜索空间可表示为各子空间规模的笛卡尔积：

$$N_{\text{all}} = N_h \cdot N_f \cdot N_p \cdot N_t \tag{4.5}$$

其中，硬件参数空间 N_h 主要由 DRAM、SRAM、数据引擎（DE）以及处理阵列（PE）的容量、带宽、并行度和流水深度等配置共同构成。具体而言，本文在 DRAM 侧考虑带宽、地址映射策略等存储系统参数，在 SRAM 侧考虑片上容

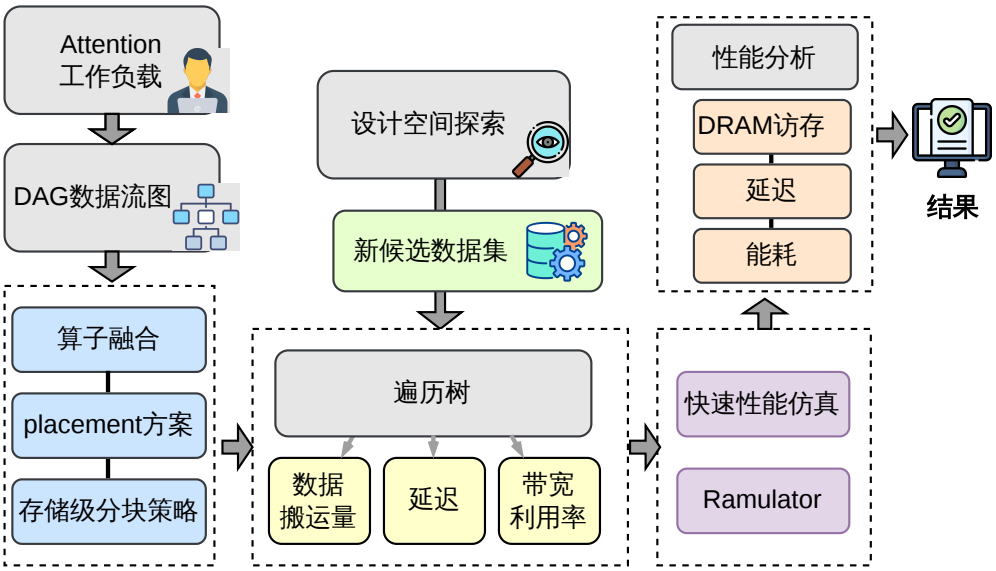


图 4.1　总体设计概览

表 4.2　联合设计空间的规模

子空间	参数维度	规模
硬件空间 N_h	DRAM、SRAM、DE 与 PE 的容量、带宽、并行度及流水相关参数	109,350
融合空间 N_f	算子融合方式、融合粒度	18
Placement 空间 N_p	模板选择、驻留策略及边界溢出	128
Tiling 空间 N_t	存储级绑定 tile 大小、循环顺序与空间映射策略	124,416
联合空间 N_{all}	四类子空间笛卡尔积	$\approx 3.13 \times 10^{13}$

量与带宽约束，在 DE 侧考虑流水深度，在 PE 侧考虑阵列规模、单周期计算能力以及输入/输出带宽。各候选值组合后，硬件空间规模为 $N_h = 3 \times 1 \times 2 \times 1 \times 3 \times 3 \times 1 \times 3 \times 3 \times 3 \times 3 \times 5 \times 5 \times 1 = 109{,}350$。

融合参数空间 N_f 用于描述 Attention 子算子在节点内的协同执行方式，主要包括融合风格、最大融合组规模以及语义角色等变量。其中，融合风格决定算子间采用顺序执行还是流水协同，最大融合组规模刻画融合粒度，特殊角色则用于区分不同功能子结构在模板选择上的约束条件。该部分空间规模为 $N_f = 2 \times 3 \times 1 \times 3 = 18$。

placement 参数空间 N_p 刻画数据与计算在层次结构中的放置方式，包括 placement 模板、执行阶段过滤、驻留策略以及写回/驱逐等动作开关。placement 维度本质上描述的是不同数据流组织方式在节点内部的实现偏好，因此其搜索并非简单枚举存放位置，而是围绕模板化策略展开。该部分空间规模为 $N_p = 4 \times 2 \times 4 \times 2 \times 2 = 128$。

tiling 参数空间 N_t 则反映循环层次划分与空间–时间映射策略，是整个设

计空间中最直接影响数据复用和并行利用效率的部分。本文将 tiling 建模为一个层次化循环分块描述，在 DRAM、SRAM 和 PE 三个层次上分别定义 tile 大小、循环顺序以及空间映射决策。依据当前候选设置，tiling 空间规模可写为 $N_t = 1 \times 1 \times 6 \times 27 \times 32 \times 12 \times 2 = 124{,}416$。

因此，在不考虑 workload 相关动态派生、group 实例化以及额外上下文约束的情况下，本文联合设计空间的理论规模为

$$N_{\text{all}} = 109{,}350 \times 18 \times 128 \times 124{,}416 \approx 3.13 \times 10^{13} \tag{4.6}$$

可以看出，即使仅采用当前离散候选集合，联合空间规模也已达到 10^{13} 量级。如此庞大的候选规模意味着，若直接采用全量穷举并对每个设计点执行完整建树、快速分析乃至 DRAM 精细仿真，将在计算成本上难以接受。因此，后续必须通过设计空间剪枝、快速代理评估以及多目标筛选等机制，对候选进行分阶段收缩和重排序，以保证探索过程在效率和精度之间取得平衡。

4.2.3　设计空间剪枝与初始化方法

由于本文构建的近存计算节点软硬件联合设计空间由硬件参数空间、融合参数空间、数据驻留参数空间以及 tiling 参数空间四部分组成，其理论规模已达到 10^{13} 量级，因此若对所有候选设计点进行全量枚举并逐一执行完整性能评估，将带来难以接受的计算开销。更为关键的是，在联合空间中，大量候选并非仅仅表现为“性能较差”，而是在给定硬件资源约束下根本不可实现。例如，某些分块策略虽然在逻辑上映射成立，但其片上缓存容量需求超过节点 SRAM 上限，或其运行所需的数据供给带宽超过 DRAM、SRAM、数据引擎及计算阵列接口所能提供的峰值能力，此类候选不应进入后续性能分析阶段。因此，本文在探索流程中首先引入设计空间剪枝机制，为形式化描述设计空间剪枝过程，本文将联合设计空间中的任一候选设计点表示为

$$\mathbf{x} = (\mathbf{h}, \mathbf{f}, \mathbf{p}, \mathbf{t}),$$

其中，$\mathbf{h}$ 表示硬件配置参数，$\mathbf{f}$ 表示融合参数，$\mathbf{p}$ 表示数据驻留与 placement 参数，$\mathbf{t}$ 表示多层次 tiling 参数。由于本文关注的是在给定硬件配置条件下的软件映射可行域构造，因此阶段 A 的输入为某一固定硬件配置 $\mathbf{h}$，其输出定义为该硬件配置所诱导的可行软件子空间

$$\Omega(\mathbf{h}) = \{(\mathbf{f}, \mathbf{p}, \mathbf{t}) \mid \Phi(\mathbf{h}, \mathbf{f}, \mathbf{p}, \mathbf{t}) = 1\}.$$

其中，$\Omega(\mathbf{h})$ 表示在硬件配置 $\mathbf{h}$ 下所有满足物理实现约束的软件候选集合，$\Phi(\mathbf{h}, \mathbf{f}, \mathbf{p}, \mathbf{t})$ 为候选设计点的可行性判定函数。当且仅当候选同时满足结构合法性约束、容量约束与带宽可满足性约束时，判定该候选属于可行子空间。故可行

性函数可进一步写为

$$\boldsymbol{\Phi}(\mathbf{h},\mathbf{f},\mathbf{p},\mathbf{t}) = \boldsymbol{\Phi}_{\mathrm{leg}}(\mathbf{h},\mathbf{f},\mathbf{p},\mathbf{t}) \wedge \boldsymbol{\Phi}_{\mathrm{cap}}(\mathbf{h},\mathbf{f},\mathbf{p},\mathbf{t}) \wedge \boldsymbol{\Phi}_{\mathrm{bw}}(\mathbf{h},\mathbf{f},\mathbf{p},\mathbf{t}),$$

其中，$\boldsymbol{\Phi}_{\mathrm{leg}}$ 表示结构合法性判定函数，用于检查候选映射在融合语义、数据驻留方式、层次分块关系以及空间映射绑定上的一致性；$\boldsymbol{\Phi}_{\mathrm{cap}}$ 表示容量约束判定函数，用于检查候选在任意执行阶段的片上数据驻留需求是否超过节点 SRAM 容量上限；$\boldsymbol{\Phi}_{\mathrm{bw}}$ 表示带宽可满足性判定函数，用于检查候选在目标执行节拍下是否能够满足 DRAM、SRAM、数据引擎及 PE 阵列接口等关键资源通路的持续供数需求。

基于上述定义，阶段 A 的本质可表述为：对于任意给定的硬件配置 $\mathbf{h}$，通过结构合法性筛选、容量约束筛选与带宽约束筛选，将原始软件映射空间收缩为仅包含物理可实现候选的可行子空间 $\Omega(\mathbf{h})$。该过程并非简单剔除低性能设计点，而是优先排除在当前资源条件下根本不可执行的非法候选，从而为后续阶段的性能优化与优选配置搜索提供约束感知的高质量搜索域。

本文所采用的剪枝过程遵循“结构合法性—容量约束—带宽可满足性”的分层过滤思路。首先，在结构合法性剪枝阶段，对候选设计点在映射语义上的有效性进行检查。该阶段主要验证候选数据驻留模式是否适用于当前 Attention 子图阶段与融合组语义，验证各层次循环分块是否满足原始问题规模限制，并检查空间映射策略与节点内部计算阵列结构是否一致。该阶段的目标是剔除定义不完整、层次绑定非法或映射关系不成立的设计点。

在结构合法性筛选之后，进入容量约束剪枝阶段。该阶段主要围绕节点内部 SRAM 的容量限制展开。对于任意候选映射方案，需要估算其在给定分块策略下于片上缓存中同时驻留的数据量，包括输入激活、键值张量、中间统计量以及部分和等数据对象。若某候选在任意执行阶段所要求的片上驻留数据规模超过 SRAM 总容量，则说明该候选无法在节点内部形成有效的数据复用与供数过程，应直接判定为不可行。容量约束剪枝的本质是保证候选设计点在静态存储资源层面可被实现，从而避免后续评估对显然无法落地的映射方案浪费仿真与分析开销。

在结构合法性筛选之后，进入容量约束剪枝阶段。对于任意候选映射方案，需要估算其在给定分块策略下于片上缓存中同时驻留的数据量，包括输入激活、键值张量、中间统计量以及部分和等数据对象。若某候选在任意执行阶段所要求的片上驻留数据规模超过 SRAM 总容量，则说明该候选无法在节点内部形成有效的数据复用与供数过程，应直接判定为不可行。容量约束剪枝的本质是保证候选设计点在静态存储资源层面可被实现，从而避免后续评估对显然无法落地的映射方案浪费仿真与分析开销。

在此基础上，本文进一步执行带宽可满足性剪枝。对于给定硬件配置和映射方案，系统需要检查节点内部多级数据通路是否能够在目标计算节拍下持续供给所需数据。分别评估节点本地 DRAM 的总带宽是否足以支撑当前分块下的数据搬运需求；DRAM Bank 的并发访问能力是否满足多流读写请求；片上 SRAM 的读写带宽是否能够持续支持缓存层与计算层之间的数据交换；以及 PE 阵列的输入带宽与输出带宽是否能够支撑对应的空间映射与并行计算过程。若任一资源通路在理论上无法满足供数要求，则说明该设计点在当前配置下无法稳定执行，应在探索早期直接剔除。

基于上述三类约束过滤后，得到一个与当前硬件配置相关联的可行子空间表示。该可行子空间保留了在当前节点资源条件下允许采用的融合方案、placement 模板集合以及对应的存储级分块策略集合，该设计使得后续搜索过程能够在更小、更干净、且具备明确物理意义的候选域中完成遍历与排序，从而提高探索效率与结果可解释性。

完成空间剪枝后，本文进一步设计了约束感知初始化方法，，从硬件参数空间中选取一组节点配置初值，包括 DRAM 存储系统参数、片上 SRAM 容量与带宽参数、数据引擎相关参数以及 PE 阵列规模和计算能力参数。随后，针对每一个硬件配置，生成对应的融合参数、placement 参数与 tiling 参数候选，并将其组合为原始搜索点。通过先固定硬件点再派生软件映射候选，可以更加清晰地反映“底层资源供给决定上层执行方式可行域”的因果关系，为后续层次化探索框架奠定了基础。

fusion 剪枝 dataplacement 剪枝 tilling 剪枝

4.2.4　层次化约束感知的设计空间探索框架

为了在设计空间探索中精确评估不同硬件配置对 Attention 推理性能的影响，本文针对基于 3D 堆叠技术的近存计算（PIM）节点构建了参数化的硬件资源模型。该模型将 PIM 节点抽象为由存储子系统、片上数据引擎（Data Engine）和计算子系统构成的异构流水线系统。近存计算节点内部的架构硬件设计需重点考量存算节点存储配置、计算单元阵列规模及片上缓存容量这三类核心参数的协同优化

在完成约束感知可行域构造后，原始联合设计空间被收缩为由硬件配置 $\mathbf{h}$ 所诱导的物理可实现子空间

$$\Omega(\mathbf{h}) = \{(\mathbf{f}, \mathbf{p}, \mathbf{t}) \mid \Phi(\mathbf{h}, \mathbf{f}, \mathbf{p}, \mathbf{t}) = 1\}.$$

因此，后续设计空间探索的目标不再是在整个原始联合空间中盲目搜索，而是转化为：在给定硬件配置 $\mathbf{h}$ 的条件下，于可行子空间 $\Omega(\mathbf{h})$ 内寻找性能最优或 Pareto 最优的软件映射方案。考虑到本文的优化目标通常同时涉及执行延迟、能耗和数

据搬运开销等多个性能指标，阶段 B 被建模为一个多目标优化问题。

具体而言，对于任一可行设计点

$$\mathbf{x} = (\mathbf{h}, \mathbf{f}, \mathbf{p}, \mathbf{t}), \qquad (\mathbf{f}, \mathbf{p}, \mathbf{t}) \in \Omega(\mathbf{h}),$$

定义其多目标代价向量为

$$\mathbf{J}(\mathbf{x}) = \left[J_1(\mathbf{x}), J_2(\mathbf{x}), \cdots, J_m(\mathbf{x})\right]^\top,$$

其中 m 表示目标数目。对于本文所关注的近存计算节点设计空间探索问题，可取

$$\mathbf{J}(\mathbf{x}) = [T(\mathbf{x}),\ E(\mathbf{x}),\ D(\mathbf{x})]^\top,$$

其中 $T(\mathbf{x})$ 表示端到端执行延迟，$E(\mathbf{x})$ 表示能耗，$D(\mathbf{x})$ 表示 DRAM 数据搬运量或总数据流量。由此，阶段 B 的优化目标可表示为

$$\min_{(\mathbf{f},\mathbf{p},\mathbf{t})\in\Omega(\mathbf{h})} \mathbf{J}(\mathbf{h}, \mathbf{f}, \mathbf{p}, \mathbf{t}),$$

其目标并非获得单一标量最优解，而是尽可能逼近该可行子空间上的 Pareto 最优解集。

由于直接在 $\Omega(\mathbf{h})$ 上进行全量多目标枚举仍然代价较高，本文采用基于 ParEGO 的多目标序贯模型优化方法。该方法的核心思想是：在每一轮迭代中，首先通过随机权重向量将多目标优化问题标量化为单目标问题；随后基于已评估样本训练代理模型（surrogate model）；最后利用采集函数（acquisition function）在可行子空间中选择下一候选点进行评估。通过“标量化—建模—选点—评估—更新”的闭环迭代，逐步逼近 Pareto 前沿。

ParEGO 标量化设当前共有 m 个优化目标，记其归一化后的目标值为

$$\tilde{J}_i(\mathbf{x}) = \frac{J_i(\mathbf{x}) - J_i^{\min}}{J_i^{\max} - J_i^{\min} + \epsilon}, \qquad i = 1, 2, \cdots, m,$$

其中 $J_i^{\min}$ 与 $J_i^{\max}$ 分别表示当前已评估样本中第 i 个目标的最小值与最大值，ϵ 为避免分母为零而引入的极小正常数。归一化步骤的目的是消除不同目标量纲差异对后续标量化结果的影响。

随后，从权重单纯形上采样一个权重向量

$$\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \cdots, \lambda_m), \qquad \lambda_i \geqslant 0, \quad \sum_{i=1}^{m} \lambda_i = 1.$$

基于该权重向量，ParEGO 采用增强 Chebyshev 标量化函数将多目标向量映射为单目标标量值

$$g_{\boldsymbol{\lambda}}(\mathbf{x}) = \max_{1\leqslant i\leqslant m} \left(\lambda_i \tilde{J}_i(\mathbf{x})\right) + \rho \sum_{i=1}^{m} \lambda_i \tilde{J}_i(\mathbf{x}),$$

其中 ρ 为一个较小的正系数，用于打破纯最大值项导致的并列情况并增强数值稳定性。通过该标量化方式，原始多目标优化问题被转化为

$$\min_{(\mathbf{f},\mathbf{p},\mathbf{t})\in\Omega(\mathbf{h})} g_{\lambda}(\mathbf{h},\mathbf{f},\mathbf{p},\mathbf{t}).$$

需要指出的是，由于每轮迭代中所采用的权重向量 λ 不同，优化过程不会固定偏向某一单一目标，而是通过在不同方向上反复搜索，逐步逼近整个 Pareto 前沿。

在第 k 轮迭代时，设已评估样本集合为

$$\mathcal{D}^{(k)} = \left\{\left(\mathbf{x}^{(j)}, \mathbf{J}(\mathbf{x}^{(j)})\right)\right\}_{j=1}^{N_k},$$

其中 N_k 表示当前已评估样本数。对给定权重向量 $\lambda^{(k)}$，首先根据上式计算每个样本的标量化值

$$y^{(j)} = g_{\lambda^{(k)}}\left(\mathbf{x}^{(j)}\right).$$

然后，利用数据集

$$\hat{\mathcal{D}}^{(k)} = \left\{\left(\mathbf{x}^{(j)}, y^{(j)}\right)\right\}_{j=1}^{N_k}$$

训练单目标代理模型 $\hat{g}^{(k)}(\mathbf{x})$，用于预测可行子空间中未评估候选点的标量化性能。考虑到本文设计空间同时包含离散变量、连续变量以及条件依赖变量，代理模型可采用适用于混合配置空间的随机森林回归器或其他非参数回归模型，以提高对复杂空间结构的建模能力。

在获得代理模型后，本文利用采集函数从可行子空间 $\Omega(\mathbf{h})$ 中选择下一候选点。设采集函数为 $a(\mathbf{x};\hat{g}^{(k)})$，则新候选点的生成可表示为

$$\mathbf{x}^{(k+1)} = \arg\max_{\mathbf{x}\in\Omega(\mathbf{h})} a(\mathbf{x};\hat{g}^{(k)}).$$

在实现上，采集函数可采用期望改进（Expected Improvement, EI）或其适用于随机森林代理的近似形式，以平衡 exploitation 与 exploration：一方面优先搜索预测性能较优的区域，另一方面保留对高不确定性区域的适度探索。

得到新候选点 $\mathbf{x}^{(k+1)}$ 后，调用性能评估模型计算其真实目标值

$$\mathbf{J}\left(\mathbf{x}^{(k+1)}\right) = \left[T\left(\mathbf{x}^{(k+1)}\right), E\left(\mathbf{x}^{(k+1)}\right), D\left(\mathbf{x}^{(k+1)}\right)\right]^{\top},$$

并将该样本加入数据集

$$\mathcal{D}^{(k+1)} = \mathcal{D}^{(k)} \cup \left\{\left(\mathbf{x}^{(k+1)}, \mathbf{J}\left(\mathbf{x}^{(k+1)}\right)\right)\right\}.$$

随后更新当前非支配解集合，并进入下一轮迭代。随着迭代不断进行，算法能够在可行子空间中逐步积累高质量设计点，并形成对 Pareto 前沿的近似。

与直接在原始联合空间中执行统一多目标搜索的方法相比，本文所采用的阶段 B 序贯模型优化方法具有三方面优势。首先，搜索范围被严格限制在阶段 A 构造得到的可行子空间 $\Omega(\mathbf{h})$ 内，因此不会将评估预算浪费在物理不可实现的

非法候选上。其次，ParEGO 通过随机权重驱动的多轮标量化搜索，能够在保持较高样本效率的同时逐步逼近 Pareto 前沿，适合评估代价较高的设计空间探索问题。最后，该方法与本文的层次化框架天然兼容：硬件参数决定可行域边界，而序贯模型优化则在该边界内部搜索最优软件映射方案，从而形成“先收缩可行域，再执行多目标优化”的两阶段探索流程。

4.3　实验结果与分析

4.3.1　性能仿真平台

由于近存计算节点的整体性能同时受到片上执行过程与片外存储访问行为的共同影响，为准确评估近存计算节点在不同执行方案下的性能表现，本文构建了一套面向片上计算与片外存储协同分析的性能仿真平台。该平台由片上解析式性能建模片外详细时序仿真两层构成。通过对 DE 缓存及 PE 阵列内部的数据搬运与计算执行过程进行快速性能估计；对片外 DRAM 访问行为进行详细时序仿真，分层统计系统的计算时延、访存代价等性能指标，为后续实验分析提供统一的性能评测基础。

在片上部分，本文采用解析式性能模型对数据装载、阵列计算和结果回写三个阶段进行建模。具体而言，平台首先依据执行方案统计各阶段在不同存储层级之间的数据传输量，得到片上数据搬运开销以及关键缓冲区的容量占用情况；随后，根据算子计算量和 PE 阵列的理论吞吐能力估计计算阶段的执行周期；在此基础上，进一步结合数据分块执行方式，模拟 load、compute 与 store 三阶段之间可能存在的流水重叠，以更接近实际执行过程。该模型能够较好反映片上阵列在不同映射和分块策略下的时延变化趋势。这一部分采用的是解析建模，其主要目标是在保证评估效率的同时，得到影响片上性能的主要瓶颈因素。

在片外部分，为更准确地刻画主存访问开销，本文引入基于访存轨迹的 DRAM 详细时序仿真流程。使用 ramulator 仿真平台估计 DRAM 的访存代价和延迟代价。将生成的 trace 输入 Ramulator/PINOS 后端进行详细时序仿真，以获得 DRAM 延迟、读写请求数量、bank 访问分布等统计结果。相比仅通过带宽上界或字节数进行粗粒度估算，详细时序仿真能够进一步反映地址映射方式、bank 并行性以及行缓冲局部性等因素对访存延迟的影响，因此可用于更加真实地表征片外存储系统对整体性能的制约。

4.3.2　基于 Ramulator 的 DRAM 仿真

本文使用 Ramulator 构建近存计算节点的 DRAM 仿真平台，以评估 PIM 节点在不同数据访问模式下的性能与能耗。Ramulator 是一个高精度、模块化的

DRAM 仿真器，支持多种 DRAM 类型，包括传统 DDR/HBM 以及近存 nDRAM。通过对时序参数、bank/row/column 并行性以及地址映射策略的建模，Ramulator 可以提供精确的访存延迟和能耗估计，同时支持与 PIM 仿真逻辑层（Logic Layer）集成。

在 nDRAM 模式下，Ramulator 能够模拟近存计算节点内部的数据流动和逻辑调度。Logic Layer 可以处理 DRAM bank 内的访问请求、跨 bank 的路由延迟以及内部并行访问冲突，从而反映真实 PIM 节点的存取特性。相比传统 DRAM，nDRAM 提供更灵活的物理布局，可自定义 bank、vault、row/column 配置，以优化 row buffer 命中率和带宽利用率。对于不同类型的算子，用户可以选择行优先（row-major）、列优先（column-major）或 tiled/strided 数据访问方式，从而更好地适配 PIM 节点的计算需求。Ramulator 为近存计算节点的性能分析提供了可靠的基础，nDRAM 模式则使得 PIM 节点的近存计算特性能够被精确模拟，从而为后续的架构设计与设计空间探索提供重要参考。

4.3.3　实验设置

4.3.4　消融实验

4.3.5　软硬件协同设计空间探索算法的效果

表 4.3　nDRAM 模型中可配置的 DRAM 参数及其含义

DRAM 参数	可选值	描述
dram_speed	nDRAM_800D, nDRAM_1600H, nDRAM_1600K	表征访问延迟和 DRAM 带宽变化。
dram_org	nDRAM_512Mb_x4, nDRAM_512Mb_x8, nDRAM_512Mb_x16	表征一次访问的数据粒度与内部组织特征。
mapping_policy	ChRaBaRoCo, RoBaRaCoCh	配置地址映射策略，表征行缓冲局部性、访问冲突以及整体有效带宽利用率。

为在设计空间搜索阶段快速评估不同 mapping 候选的能耗代价，本文采用基于访问量与计算量的分层加权能耗估算方法，将总能耗近似分解为存储能耗与计算能耗两部分，即

$$E_{\text{total}} \approx E_{\text{memory}} + E_{\text{compute}}.$$

其中，存储能耗采用分层累计的方式进行粗粒度估算。设系统包含的存储层次集合为 $\mathcal{L}$，对任意存储层 $l \in \mathcal{L}$，分别统计其读访问字节数与写访问字节数，

记为 $\mathrm{BytesRead}_l$ 和 $\mathrm{BytesWrite}_l$；同时，设该层单位字节读能耗与单位字节写能耗分别为 e_l^{read} 和 e_l^{write}，则存储能耗可表示为

$$E_{\mathrm{memory}} = \sum_{l \in \mathcal{L}} \left(\mathrm{BytesRead}_l \cdot e_l^{\mathrm{read}} + \mathrm{BytesWrite}_l \cdot e_l^{\mathrm{write}} \right).$$

上述模型能够自然覆盖不同层级的存储结构，例如 DRAM、片上 SRAM 以及 PE-local buffer。对于每一层存储，本文仅需统计数据读写总量，并结合相应的单位访问能耗参数，即可得到粗粒度但具有可比性的 memory energy 估计值。该建模方式避免了在搜索早期显式构建完整访存时序，从而能够以较低代价完成对候选解的快速评价。

进一步地，该能耗模型与本文的快速剪枝目标相一致：在 tree-based analysis 阶段，仅基于访问数据量与执行代价对候选进行快速估算，用于尽早剔除明显劣势的 mapping；而对于保留下来的少量高质量候选，则在后续阶段结合更精细的 trace 生成与 Ramulator 仿真，对其真实 DRAM cycles 与能耗进行进一步确认。因而，上述分层加权模型主要服务于搜索前期的高效筛选，而非替代后期的精确性能分析。

4.4　本章小结

参考文献

[1] Sengar S S, Hasan A B, Kumar S, et al. Generative artificial intelligence: a systematic review and applications[J]. Multimedia Tools and Applications, 2025, 84(21): 23661-23700.

[2] Chen L, Li S, Bai Q, et al. Review of image classification algorithms based on convolutional neural networks[J]. Remote Sensing, 2021, 13(22).

[3] Sneha, Kaul A. Hyperspectral imaging and target detection algorithms: a review[J]. Multimedia Tools and Applications, 2022, 81(30): 44141-44206.

[4] Dosovitskiy A, Beyer L, Kolesnikov A, et al. An image is worth 16x16 words: Transformers for image recognition at scale[C]//International Conference on Learning Representations. 2021.

[5] Yu J, de Antonio A, Villalba-Mora E. Deep learning (cnn, rnn) applications for smart homes: A systematic review[J]. Computers, 2022, 11(2).

[6] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Guyon I, Luxburg U V, Bengio S, et al. Advances in Neural Information Processing Systems: Vol. 30. Curran Associates, Inc., 2017.

[7] Ren X, Zhou P, Meng X, et al. Pangu-□: Towards trillion parameter language model with sparse heterogeneous computing[A]. 2023. arXiv: 2303.10845.

[8] Chang E, Paltenghi M, Li Y, et al. Scaling parameter-constrained language models with quality data[C]//Dernoncourt F, Preoţiuc-Pietro D, Shimorina A. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track. Miami, Florida, US: Association for Computational Linguistics, 2024: 80-97.

[9] Chung H W, Hou L, Longpre S, et al. Scaling instruction-finetuned language models[J]. Journal of Machine Learning Research, 2024, 25(70): 1-53.

[10] Team G, Anil R, Borgeaud S, et al. Gemini: A family of highly capable multimodal models [A]. 2025. arXiv: 2312.11805.

[11] OpenAI, et al. Gpt-4 technical report[A]. 2024. arXiv: 2303.08774.

[12] Goertzel B. Artificial General Intelligence: Concept, State of the Art, and Future Prospects [J]. Journal of Artificial General Intelligence, 2014, 5(1): 1-48.

[13] Isaev M, McDonald N, Vuduc R. Scaling infrastructure to support multi-trillion parameter LLM training[C]//Architecture and System Support for Transformer Models (ASSYST @ISCA 2023). 2023.

[14] Duman Keles F, Wijewardena P M, Hegde C. On the computational complexity of self-

attention[C]//Agrawal S, Orabona F. Proceedings of Machine Learning Research: Vol. 201 Proceedings of The 34th International Conference on Algorithmic Learning Theory. PMLR, 2023: 597-619.

[15] Reidy B C, Mohammadi M, Elbtity M E, et al. Efficient deployment of transformer models on edge TPU accelerators: A real system evaluation[C]//Architecture and System Support for Transformer Models (ASSYST @ISCA 2023). 2023.

[16] Xu B, Banerjee A, Gupta S. Hardware acceleration for neural networks: A comprehensive survey[A]. 2026. arXiv: 2512.23914.

[17] Tay Y, Dehghani M, Bahri D, et al. Efficient transformers: A survey[J]. ACM Comput. Surv., 2022, 55(6).

[18] Zhuang Y, Zhang J, Tu M. Long-range sequence modeling with predictable sparse attention[C]//Muresan S, Nakov P, Villavicencio A. Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Dublin, Ireland: Association for Computational Linguistics, 2022: 234-243.

[19] Zhu C, Ping W, Xiao C, et al. Long-short transformer: Efficient transformers for language and vision[C]//Ranzato M, Beygelzimer A, Dauphin Y, et al. Advances in Neural Information Processing Systems: Vol. 34. Curran Associates, Inc., 2021: 17723-17736.

[20] Ootomo H, Yokota R. Reducing shared memory footprint to leverage high throughput on tensor cores and its flexible api extension library[C]//HPCAsia ’23: Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region. New York, NY, USA: Association for Computing Machinery, 2023: 1–8.

[21] Wulf W A, McKee S A. Hitting the memory wall: implications of the obvious[J]. SIGARCH Comput. Archit. News, 1995, 23(1): 20–24.

[22] Gholami A, Yao Z, Kim S, et al. Ai and memory wall[J]. IEEE Micro, 2024, 44(3): 33-39.

[23] Wu R, Zhou S, Lu J, et al. Removing obstacles before breaking through the memory wall: A close look at HBM errors in the field[C]//2024 USENIX Annual Technical Conference (USENIX ATC 24). Santa Clara, CA: USENIX Association, 2024: 851-867.

[24] Zhao X, Wang L, Zhang Y, et al. A review of convolutional neural networks in computer vision[J]. Artificial Intelligence Review, 2024, 57(4): 99.

[25] Nie Q, Malik S. Cnnflow: Memory-driven data flow optimization for convolutional neural networks[J]. ACM Trans. Des. Autom. Electron. Syst., 2023, 28(3).

[26] Liu Y, Zhang Y, Hao X, et al. Design of a convolutional neural network accelerator based on on-chip data reordering[J]. Electronics, 2024, 13(5).

[27] Kwon W, Li Z, Zhuang S, et al. Efficient memory management for large language model serving with pagedattention[C]//SOSP ’23: Proceedings of the 29th Symposium on Operating

Systems Principles. New York, NY, USA: Association for Computing Machinery, 2023: 611–626.

[28] Dao T, Fu D, Ermon S, et al. Flashattention: Fast and memory-efficient exact attention with io-awareness[C]//Koyejo S, Mohamed S, Agarwal A, et al. Advances in Neural Information Processing Systems: Vol. 35. Curran Associates, Inc., 2022: 16344-16359.

[29] Micikevicius P, Stosic D, Burgess N, et al. Fp8 formats for deep learning[A]. 2022. arXiv: 2209.05433.

[30] Chen C, Borgeaud S, Irving G, et al. Accelerating large language model decoding with speculative sampling[A]. 2023. arXiv: 2302.01318.

[31] Ansari M Q, Ansari M Q. Racing to idle: Energy efficiency of matrix multiplication on heterogeneous cpu and gpu architectures[A]. 2025. arXiv: 2507.20063.

[32] Sanders J, Kandrot E. CUDA by Example: An Introduction to General-Purpose GPU Programming[M]. Pearson Education, 2010.

[33] Wu X, Li C, Yazdani Aminabadi R, et al. Understanding int4 quantization for language models: Latency speedup, composability, and failure cases[C]//Krause A, Brunskill E, Cho K, et al. Proceedings of Machine Learning Research: Vol. 202 Proceedings of the 40th International Conference on Machine Learning. PMLR, 2023: 37524-37539.

[34] Kang B J, Lee H I, Yoon S K, et al. A survey of fpga and asic designs for transformer inference acceleration and optimization[J]. Journal of Systems Architecture, 2024, 155: 103247.

[35] Liu F, Li H, Hu W, et al. Review of neural network model acceleration techniques based on fpga platforms[J]. Neurocomputing, 2024, 610: 128511.

[36] Lai Y H, Ustun E, Xiang S, et al. Programming and synthesis for software-defined fpga acceleration: Status and future prospects: Vol. 14[M]. New York, NY, USA: Association for Computing Machinery, 2021.

[37] Jouppi N, Kurian G, Li S, et al. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings[C]//ISCA '23: Proceedings of the 50th Annual International Symposium on Computer Architecture. New York, NY, USA: Association for Computing Machinery, 2023.

[38] Huawei atlas ai computing solution[M]//Artificial Intelligence Technology. Singapore: Springer Nature Singapore, 2023: 163-219.

[39] Xu R, Ma S, Guo Y, et al. A survey of design and optimization for systolic array-based dnn accelerators[J]. ACM Comput. Surv., 2023, 56(1).

[40] Seshadri V, Lee D, Mullins T, et al. Ambit: in-memory accelerator for bulk bitwise operations using commodity dram technology[C]//MICRO-50 '17: Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture. New York, NY, USA: Associa-

tion for Computing Machinery, 2017: 273–287.

[41] Mutlu O, Ghose S, Gómez-Luna J, et al. Processing data where it makes sense: Enabling in-memory computation[J]. Microprocessors and Microsystems, 2019, 67: 28-41.

[42] Oliveira G F, Gómez-Luna J, Ghose S, et al. Methodologies, workloads, and tools for processing-in-memory: Enabling the adoption of data-centric architectures[C]//2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI). 2022: 261-266.

[43] Pawlowski J T. Hybrid memory cube (hmc)[C]//2011 IEEE Hot Chips 23 Symposium (HCS). 2011: 1-24.

[44] Kim J S, Oh C S, Lee H, et al. A 1.2 v 12.8 gb/s 2 gb mobile wide-i/o dram with 4 × 128 i/os using tsv based stacking[J]. IEEE Journal of Solid-State Circuits, 2012, 47(1): 107-116.

[45] Jun H, Cho J, Lee K, et al. Hbm (high bandwidth memory) dram technology and architecture [C]//2017 IEEE International Memory Workshop (IMW). 2017: 1-4.

[46] Niu D, Li S, Wang Y, et al. 184qps/w 64mb/mm 2 3d logic-to-dram hybrid bonding with process-near-memory engine for recommendation system[C]//2022 IEEE International Solid-State Circuits Conference (ISSCC): Vol. 65. IEEE, 2022: 1-3.

[47] Niu D, Li S, Wang Y, et al. 184qps/w 64mb/mm23d logic-to-dram hybrid bonding with process-near-memory engine for recommendation system[C]//2022 IEEE International Solid-State Circuits Conference (ISSCC): Vol. 65. 2022: 1-3.

[48] Lam M S, Wolf M E. A data locality optimizing algorithm[J]. SIGPLAN Not., 2004, 39(4): 442–459.

[49] Kennedy K, Allen J R. Optimizing compilers for modern architectures: a dependence-based approach[M]. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2001.

[50] Cheng J. Wrox Programmer to Programmer: Professional CUDA C programming[M]. Indianapolis (Indiana): John Wiley and Sons, 2014.

[51] Kirk D, Hwu W m. Programming massively parallel processors : a hands-on approach[M]. Third edition ed. Cambridge, MA, United States: Morgan Kaufmann, an imprint of Elsevier, 2017.

[52] Asifuzzaman K, Miniskar N R, Young A R, et al. A survey on processing-in-memory techniques: Advances and challenges[J]. Memories - Materials, Devices, Circuits and Systems, 2023, 4: 100022.

[53] Du X, Yu T, Zhou H, et al. ROM-SRAM hybrid compute-in-memory for edge AI: circuits, architectures and challenges[J]. The Journal of Supercomputing, 2025, 81(17): 1606.

[54] Parashar A, Raina P, Shao Y S, et al. Timeloop: A systematic approach to dnn accelerator evaluation[C]//2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS). 2019: 304-315.

[55] Zheng S, Chen S, Gao S, et al. Tileflow: A framework for modeling fusion dataflow via tree-based analysis[C]//MICRO '23: Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. New York, NY, USA: Association for Computing Machinery, 2023: 1271–1288.

[56] Ragan-Kelley J, Barnes C, Adams A, et al. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines[C]//PLDI '13: Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation. New York, NY, USA: Association for Computing Machinery, 2013: 519–530.

[57] Chen T, Moreau T, Jiang Z, et al. TVM: An automated End-to-End optimizing compiler for deep learning[C]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). Carlsbad, CA: USENIX Association, 2018: 578-594.

[58] Tillet P, Kung H T, Cox D. Triton: an intermediate language and compiler for tiled neural network computations[C]//MAPL 2019: Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages. New York, NY, USA: Association for Computing Machinery, 2019: 10–19.

[59] Prabhakar R, Jairath S, Shin J L. Sambanova sn10 rdu: A 7nm dataflow architecture to accelerate software 2.0[C]//2022 IEEE International Solid-State Circuits Conference (ISSCC): Vol. 65. 2022: 350-352.

[60] Heo G, Lee S, Cho J, et al. Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing[C]//ASPLOS '24: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. New York, NY, USA: Association for Computing Machinery, 2024: 722–737.

[61] Li W, Sun X, Wang X, et al. Pimsyn: Synthesizing processing-in-memory cnn accelerators [C]//2024 Design, Automation and Test in Europe Conference and Exhibition (DATE). 2024: 1-6.

[62] Reagen B, Hernández-Lobato J M, Adolf R, et al. A case for efficient accelerator design space exploration via bayesian optimization[C]//2017 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED). 2017: 1-6.

[63] Kwon H, Chatarasi P, Pellauer M, et al. Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach[C]//MICRO-52: Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture. New York, NY, USA: Association for Computing Machinery, 2019: 754–768.

[64] Yang X, Gao M, Liu Q, et al. Interstellar: Using halide's scheduling language to analyze dnn accelerators[C]//ASPLOS '20: Proceedings of the Twenty-Fifth International Conference on

Architectural Support for Programming Languages and Operating Systems. New York, NY, USA: Association for Computing Machinery, 2020: 369–383.

[65] Kao S C, Krishna T. Gamma: automating the hw mapping of dnn models on accelerators via genetic algorithm[C]//ICCAD '20: Proceedings of the 39th International Conference on Computer-Aided Design. New York, NY, USA: Association for Computing Machinery, 2020.

[66] Kao S C, Jeong G, Krishna T. Confuciux: Autonomous hardware resource assignment for dnn accelerators using reinforcement learning[C]//2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). 2020: 622-636.

[67] Xu P, Zhang X, Hao C, et al. Autodnnchip: An automated dnn chip predictor and builder for both fpgas and asics[C]//FPGA '20: Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York, NY, USA: Association for Computing Machinery, 2020: 40–50.

[68] Mei L, Houshmand P, Jain V, et al. Zigzag: Enlarging joint architecture-mapping design space exploration for dnn accelerators[J]. IEEE Transactions on Computers, 2021, 70(8): 1160-1174.

[69] Qi Y, Zhang S, Taha T M. Trim: A design space exploration model for deep neural networks inference and training accelerators[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2023, 42(5): 1648-1661.

[70] Feng K, Fan X, An J, et al. Csdse: An efficient design space exploration framework for deep neural network accelerator based on cooperative search[J]. Neurocomputing, 2025, 623: 129366.

[71] 王君鹏. 面向存算融合架构的神经网络映射与设计空间探索方法[D]. 中国科学技术大学, 2024.

[72] Davies M, Srinivasa N, Lin T H, et al. Loihi: A neuromorphic manycore processor with on-chip learning[J]. IEEE Micro, 2018, 38(1): 82-99.

[73] Agrawal A, Kedia N, Panwar A, et al. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve[C]//18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). Santa Clara, CA: USENIX Association, 2024: 117-134.

[74] Pope R, Douglas S, Chowdhery A, et al. Efficiently scaling transformer inference[C]//Song D, Carbin M, Chen T. Proceedings of Machine Learning and Systems: Vol. 5. Curan, 2023: 606-624.

[75] Yu G I, Jeong J S, Kim G W, et al. Orca: A distributed serving system for Transformer-Based generative models[C]//16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). Carlsbad, CA: USENIX Association, 2022: 521-538.

[76] Kwon W, Li Z, Zhuang S, et al. Efficient memory management for large language model

serving with pagedattention[C]//SOSP ’23: Proceedings of the 29th Symposium on Operating Systems Principles. New York, NY, USA: Association for Computing Machinery, 2023: 611–626.

[77] Shazeer N. Fast transformer decoding: One write-head is all you need[A]. 2019. arXiv: 1911.02150.

[78] Ainslie J, Lee-Thorp J, de Jong M, et al. GQA: Training generalized multi-query transformer models from multi-head checkpoints[C]//Bouamor H, Pino J, Bali K. Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Singapore: Association for Computational Linguistics, 2023: 4895-4901.

[79] Williams S, Waterman A, Patterson D. Roofline: an insightful visual performance model for multicore architectures[J]. Commun. ACM, 2009, 52(4): 65–76.

[80] Sze V, Chen Y H, Yang T J, et al. Synthesis lectures on computer architecture: Efficient processing of deep neural networks[M]. Springer Cham, 2020: 254.

[81] Jouppi N P, Young C, Patil N, et al. In-datacenter performance analysis of a tensor processing unit[J]. SIGARCH Comput. Archit. News, 2017, 45(2): 1–12.

[82] Chen Y H, Krishna T, Emer J S, et al. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks[J]. IEEE Journal of Solid-State Circuits, 2017, 52(1): 127-138.

[83] Horowitz M. 1.1 computing’s energy problem (and what we can do about it)[C]//2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC). 2014: 10-14.

[84] Deb K, Deb K. Multi-objective optimization[M]//Burke E K, Kendall G. Search Methodologies: Introductory Tutorials in Optimization and Decision Support Techniques. Boston, MA: Springer US, 2014: 403-449.

[85] Miettinen K. International series in operations research & management science; 12; 12.: Nonlinear multiobjective optimization[M]. Boston: Kluwer Academic Publishers, 1999.

[86] Ehrgott M. Multicriteria optimization[M]. Berlin, Heidelberg: Springer-Verlag, 2005.

[87] Kirkpatrick S, Gelatt C D, Vecchi M P. Optimization by simulated annealing[J]. Science, 1983, 220(4598): 671-680.

[88] Holland J H. Complex adaptive systems.: Adaptation in natural and artificial systems : an introductory analysis with applications to biology, control, and artificial intelligence[M]. 4th printing ed. Cambridge, Massachusetts: MIT Press, 1995.

[89] Glover F. Tabu search—part i[J]. Orsa J on Computing, 1989, 1(1): 89-98.

[90] Kennedy J, Eberhart R. Particle swarm optimization[C]//Proceedings of ICNN’95 - International Conference on Neural Networks: Vol. 4. 1995: 1942-1948 vol.4.

[91] Goldberg D E. Genetic algorithms in search, optimization, and machine learning[M]. 22nd printing ed. Boston [etc.]: Addison-Wesley, 2001.

[92] SEEGER M. Gaussian processes for machine learning[J]. International Journal of Neural Systems, 2004, 14(02): 69-106.

[93] Jones D R, Schonlau M, Welch W J. Efficient Global Optimization of Expensive Black-Box Functions[J]. Journal of Global Optimization, 1998, 13(4): 455-492.

[94] Daulton S, Balandat M, Bakshy E. Differentiable expected hypervolume improvement for parallel multi-objective bayesian optimization[C]//Larochelle H, Ranzato M, Hadsell R, et al. Advances in Neural Information Processing Systems: Vol. 33. Curran Associates, Inc., 2020: 9851-9864.

致谢

在读期间取得的科研成果